

Business Process Automation with VBA and Python

Mr. Eddie Chow / 30 May 2026

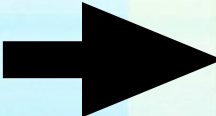


Table Of Contents

Introduction to business process automation

Business Process automation with VBA

Business process automation with Python



Introduction to project management for business process automation

Development and implementation of business process automation

Final Group Presentation





What is Process Automation, RPA, IPA?
Overview of the technological building blocks related to process automation
Contemporary tools for process automation
Challenges and opportunities of business process automation
Business implications of process automation



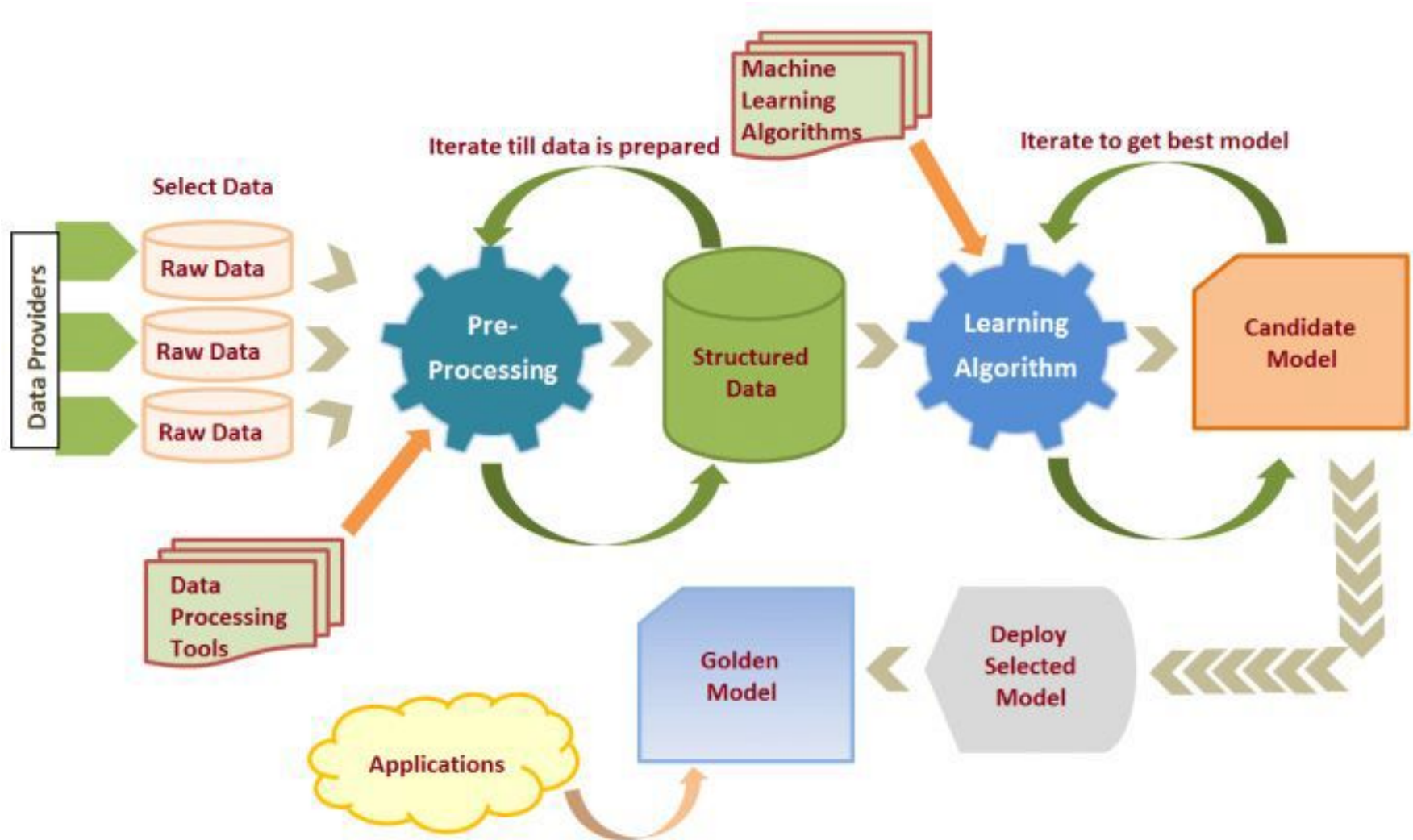
Intended Learning Outcomes

1. describe the contemporary trends of process automation and explain the opportunities and challenges of business process automation;
2. outline key steps in process automation project management and illustrate the significance of each step;
3. apply computational tools to implement process automation;
4. discuss the development of process automation and practical cases for business.

Agenda

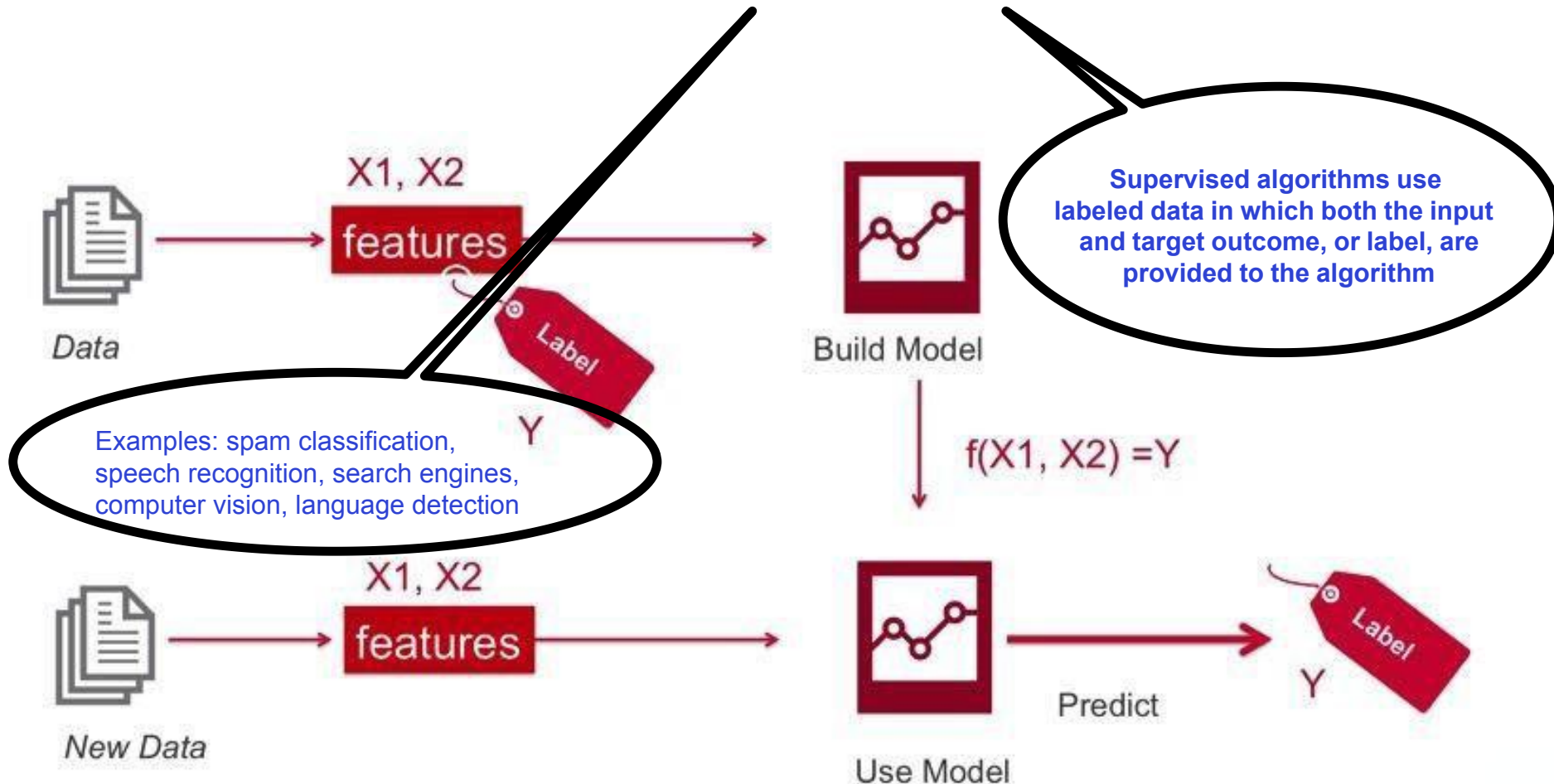
1. Exploratory Data Analysis(ETL), Dynamic Data Visualization
2. Financial Forecasting with Machine Learning using Python
3. Introduction to Project Management in Business Process Automation
4. Methodologies in Project Management
5. Key Steps in Project Management
6. Understanding Business Process Automation
7. Integration of Project Management and Business Process Automation
8. Organizational Issues in Business Process Automation
9. Projects 7 Project Scope and Requirements Gathering
10. Risk Management in Automation Projects
11. Change Management Best Practices
12. Performance Measurement and Key Performance Indicators (KPIs)
13. Case Studies of Successful Automation Projects
14. Conclusion and Future Directions

Applied Machine Learning Pipeline Overview



Data Processing & Analysis

Machine Learning: Supervised Learning



Source:
<https://dzone.com/articles/demystifying-ai-machine-learning-and-deep-learning>

Data Processing & Analysis

Machine Learning: Decision Trees

decision
tree for predicting debit
card fraud

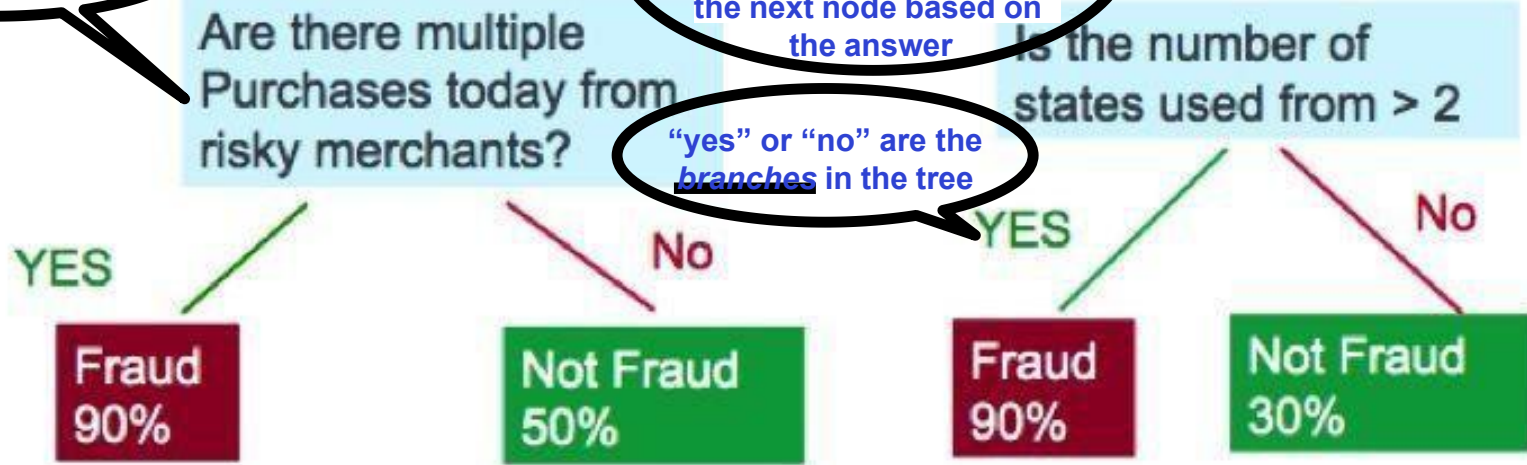
Decision trees create a model that
predicts the class or label based on several
input features

questions are the nodes

Is the amount spent in 24
hours > average

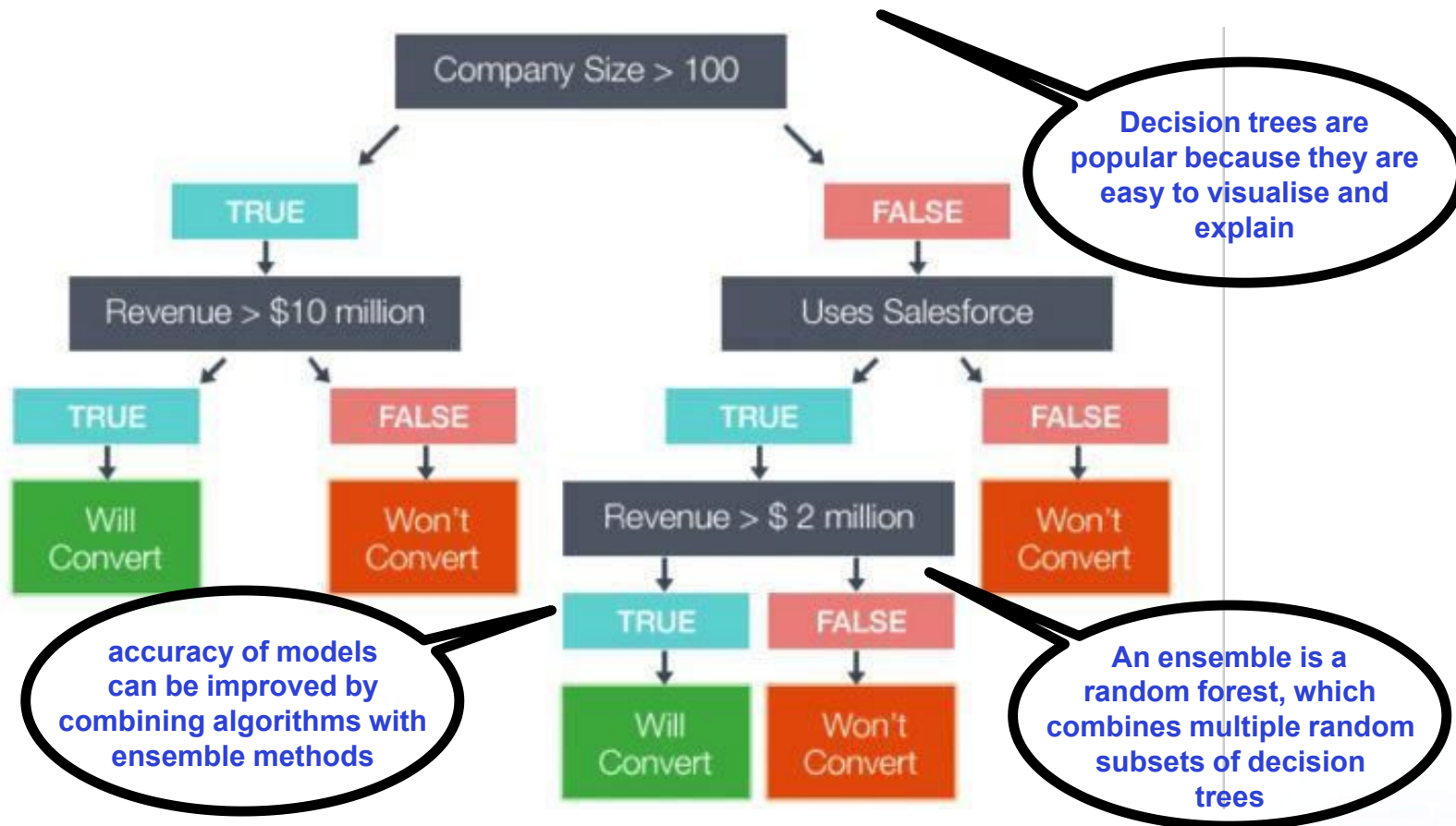
work
by evaluating a question
containing a feature at every
node and selecting a branch to
the next node based on
the answer

“yes” or “no” are the
branches in the tree



Data Processing & Analysis

Decision Trees: Ensemble Random Forests



Source: <http://dataconomy.com/2015/05/predictive-machine-learning-behind-the-scenes-at-fliptop-and-predictions-for-the-future-of-martech/>

How To Do Data Mining: Machine Learning Techniques

Features / Attributes

Set X using RandomForestRegressor
Parameter

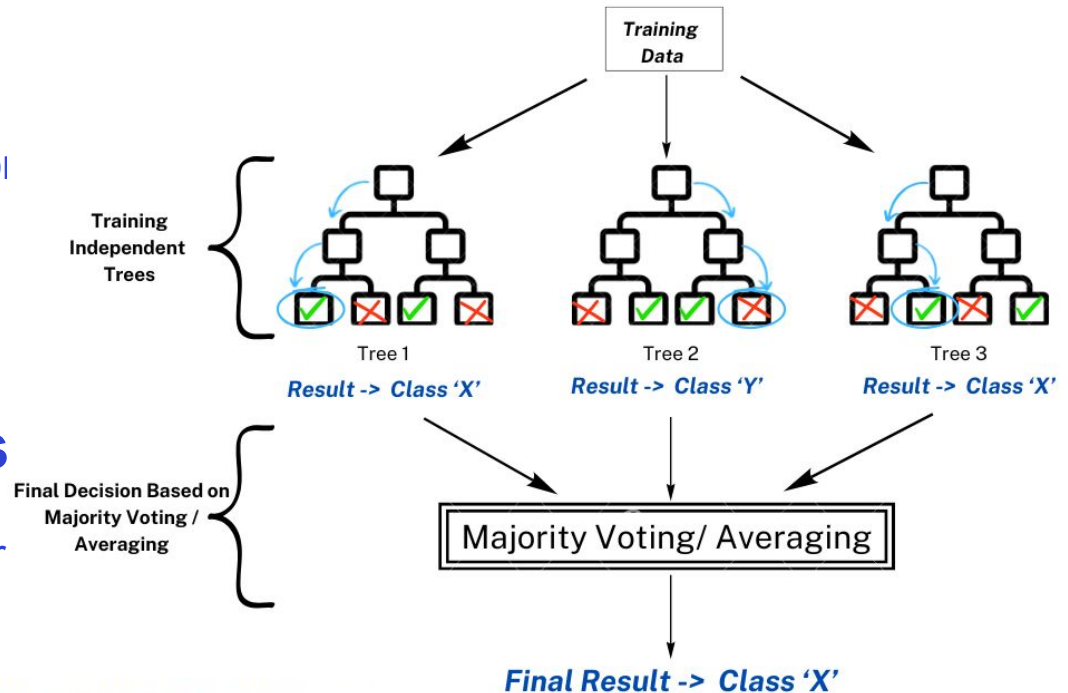
1. District
2. Floor
3. School Net Numbers

Set y using RandomForestRegressor
Outcome - Price Per Square

Top 5 Features and their Importance Scores:

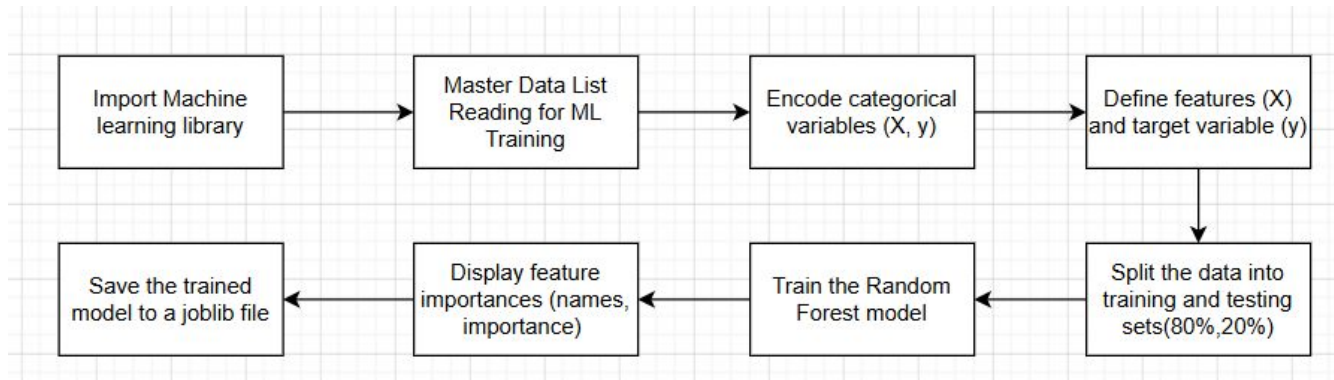
	Feature	Importance
0	District	0.728459
2	School_Netnumber	0.199190
1	Floor	0.072351

Random Forest Algorithm in Machine Learning



How To Do Data Mining: Machine Learning Techniques

Below are the Flow Chart with the Process of Machine Learning Random Forest Model Training



#This code is going to train an random forest model to find top important features

#Step 1 : Import necessary library

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestRegressor
from sklearn.tree import plot_tree
import joblib # Import joblib for saving the model
```

Machine Learning Random Forest Model training

#Step 2 : Master Data List Reading for ML Training

```
data = pd.read_csv("combined_building_withweather_school.csv")
```

#Step 3 : Encode categorical variables (X, y)

```
label_encoder = LabelEncoder()
```

```
data['District'] = label_encoder.fit_transform(data['District'])
```

```
data['School_Netnumber'] = label_encoder.fit_transform(data['School_Netnumber'])
```

#Step 4: Define features and target variable

```
X = data[['District', 'Floor', 'School_Netnumber']] # Features
```

```
y = data['Price Per Square'] # Target variable
```

```
# Check the distribution of the target variable
```

```
print(y.value_counts())
```

```
print(X)
```

```
# Ensure the target variable is continuous
```

```
print(f"Target variable type: {y.dtype}")
```

```
Price Per Square
0.00      30508
10000.00    217
8333.33    109
8000.00     81
12500.00     70
...
8554.05      1
13764.04      1
10443.86      1
15421.30      1
4672.90       1
Name: count, Length: 31331, dtype: int64
   District  Floor  School_Netnumber
0         1    2.0             8
1        16    1.0             2
2        31    7.0             5
3        28    1.0             7
4        24    1.0             2
...      ...    ...             ...
79995      6    2.0             9
79996      9    9.0             6
79997      3    4.0             4
79998      6    1.0             9
79999     26    3.0             1
[80000 rows x 3 columns]
```

Machine Learning Random Forest Model training

#Step 5: Split the dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

#Step 6: Train the Random Forest model

```
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
```

```
rf_model.fit(X_train, y_train)
```

#Display a decision tree from the Random Forest with reduced size

```
plt.figure(figsize=(6, 6)) # Reduced by 10%
```

```
plot_tree(rf_model.estimators_[5], feature_names=X.columns.tolist(), filled=True)
```

```
plt.title("Decision Tree from Random Forest")
```

```
plt.show()
```

```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

Machine Learning Random Forest Model training

#Step 8: Display feature importances

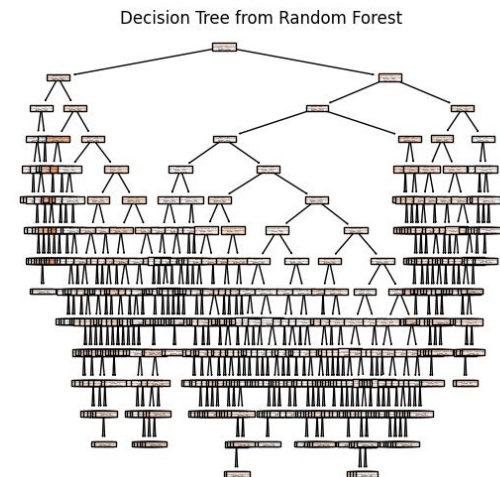
```
importances = rf_model.feature_importances_  
feature_names = X.columns.tolist()  
feature_importances = pd.DataFrame({'Feature': feature_names, 'Importance':  
    importances})
```

Sort by importance and display the top 5 features

```
top_features = feature_importances.sort_values(by='Importance',  
    ascending=False).head(5)  
print("Top 5 Features and their Importance Scores:")  
print(top_features)
```

#Step 9: Save the trained model to a joblib file

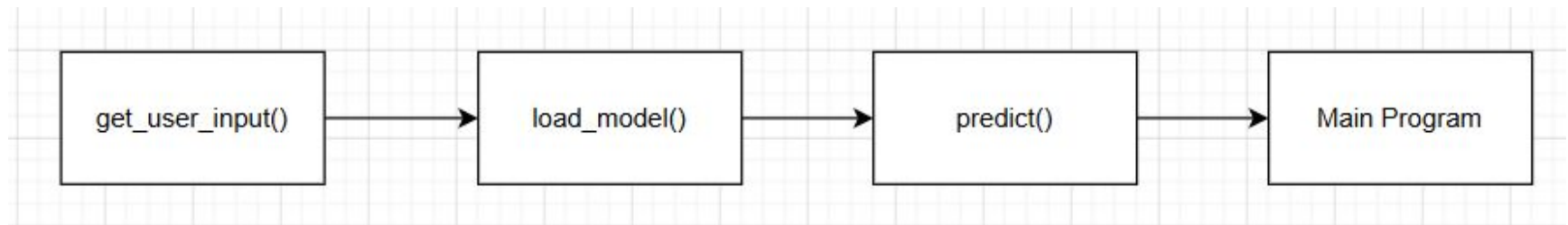
```
model_filename = 'housing_valuation_model.joblib' # Specify the filename  
joblib.dump(rf_model, model_filename) # Save the model  
print(f'Model saved to {model_filename}')
```



Top 5 Features and their Importance Scores:

	Feature	Importance
0	District	0.728459
2	School_Netnumber	0.199190
1	Floor	0.072351

Machine Learning Random Forest Inference Model : Display the predicted housing price per square



```
#add necessary library
import joblib
```

```
#Define function to get user input
```

```
def get_user_input():
    district = float(input("Enter District Number : "))
    schoolnet_no = float(input("Enter School Net Number: "))
    floor = float(input("Enter Floor "))
    input_list = [[district, schoolnet_no, floor]] #assign in data type {list}
    return input_list
```

Machine Learning Random Forest Inference Model : Display the predicted housing price per square

```
#Define function for loading random forest model
def load_model(model_path):
    return joblib.load(model_path)

#Define function to predict housing price per square based on 3 data input
def predict(model, input_data):
    return model.predict(input_data)

#Main Program to display the predicted housing price per square
input_data = get_user_input()
model_path = 'housing_valuation_model.joblib'
model = load_model(model_path)
result = predict(model, input_data)
print(f"Predicted Hosing Price Per Square is : HKD ${round(result[0],2)}")
```

```
Enter District Number : 12
```

```
Enter School Net Number: 24
```

```
Enter Floor 3
```

```
Predicted Hosing Price Per Square is : HKD $10839.77
```

Business Process Automation - Data Type “Dict”

- **API endpoint used**

- URL:
<https://data.weather.gov.hk/weatherAPI/opendata/weather.php?dataType=flw&lang=tc>
- Purpose: retrieves **local weather forecast** (flw) in **Traditional Chinese** (lang=tc)

- **Sample response structure**

- Example `weather_data` object:

```
weather_data = {  
    "generalSituation": "Mainly cloudy with a few showers",  
    "temperature": {  
        "value": 28,  
        "unit": "C"  
    },  
    "humidity": 85,  
    "wind": "Moderate southerly winds",  
    "updateTime": "2026-05-20 12:00"  
}
```

Dict Data Type

```
print(weather_data["temperature"])  
print(weather_data["temperature"]["value"])  
print(weather_data["humidity"])
```

Business Process Automation with Python

Data Type - Dictionary

- Representation

```
# Representation: option 1
school_rank_dict = {'hku': 21, 'cuhk': 38, 'hkust': 40}
print(type(school_rank_dict))

# Representation: option 2
school_rank_dict = dict(hku=21, cuhk=38, hkust=40)
print(school_rank_dict)
```

<class 'dict'>
{'hku': 21, 'cuhk': 38, 'hkust': 40}

+ Rank	- University
21	 The University of Hong Kong Hong Kong, Hong Kong SAR
38	 The Chinese University of Hong Kong (CUHK) Hong Kong SAR, Hong Kong SAR
40	 The Hong Kong University of Science and Technology Hong Kong SAR, Hong Kong SAR

- Lookup → get()

```
# Lookup
print(school_rank_dict['cuhk'])           # Direct lookup
print(school_rank_dict.get('hkust'))      # get(): Success
print(school_rank_dict.get('test'))       # get(): Failed
print(school_rank_dict.get('test', 'N/A')) # get(): Error handling
```

38
40
None
N/A

Business Process Automation with Python

Data Type - Dictionary

- Adding an item

```
# Adding an item
school_rank_dict = {'hku': 21, 'cuhk': 38, 'hkust': 40}
school_rank_dict['cityu'] = 54
print(school_rank_dict)
```

{'hku': 21, 'cuhk': 38, 'hkust': 40, 'cityu': 54}

- Removing an item

```
# Removing an item
school_rank_dict = {'hku': 21, 'cuhk': 38, 'hkust': 40, 'cityu': 54}
school_rank_dict.pop('cityu')
print(school_rank_dict)
```

{'hku': 21, 'cuhk': 38, 'hkust': 40}

Business Process Automation with Python

Data Type - Dictionary

- Listing out the details

```
# Show all the keys
print(f'keys: {list(school_rank_dict.keys())}')

# Show all the values
print(f'values: {list(school_rank_dict.values())}')

# Show all the items
print(f'items: {list(school_rank_dict.items())}')

keys: ['hku', 'cuhk', 'hkust']
values: [21, 38, 40]
items: [('hku', 21), ('cuhk', 38), ('hkust', 40)]
```

- Membership Check

```
# Membership Check
print('hku' in school_rank_dict)

True
```

Business Process Automation with Python

Loops – Using “for”

- Looping through a dictionary

```
▶ school_rank_dict = {'hku': 21, 'cuhk': 38, 'hkust': 40}

for school, rank in school_rank_dict.items():
    print(f'{school} is ranked {rank} in the world!')
```

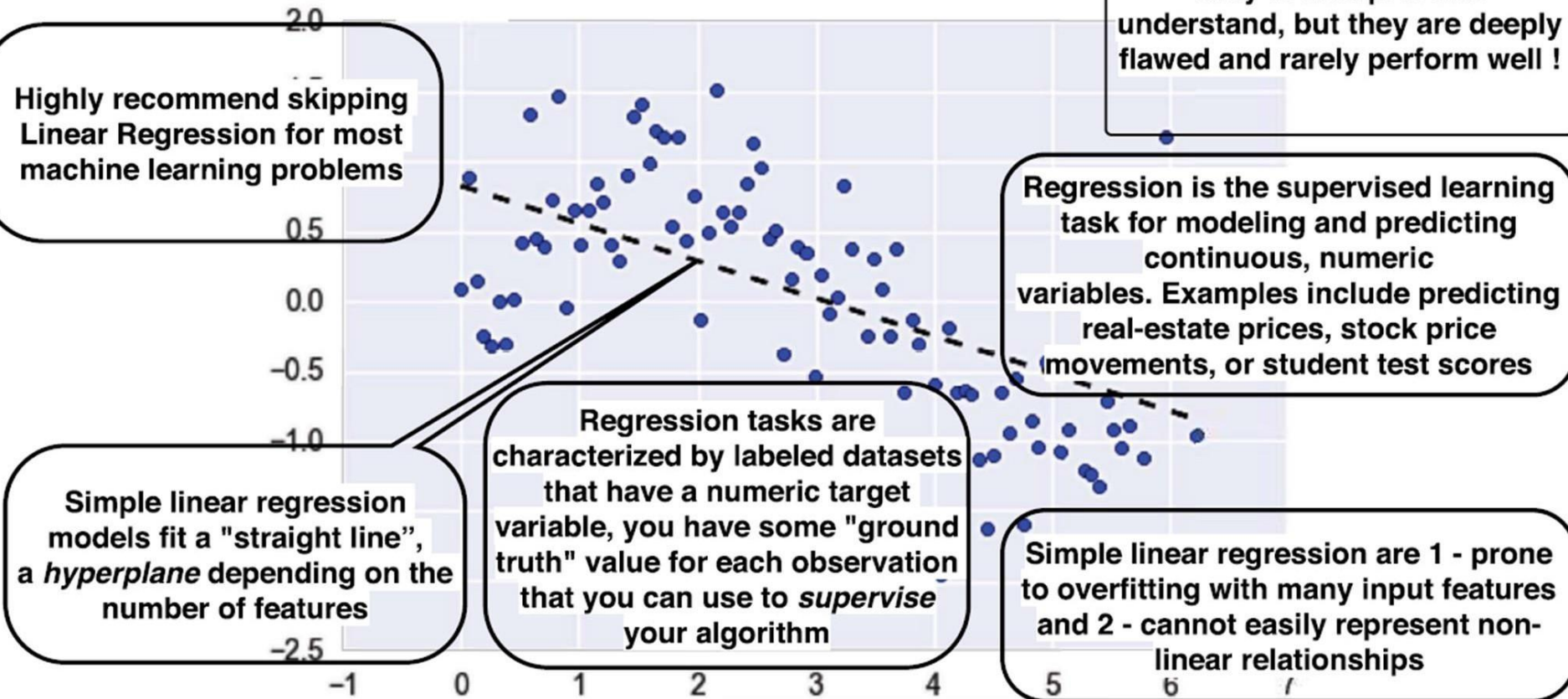
```
↳ hku is ranked 21 in the world!
   cuhk is ranked 38 in the world!
   hkust is ranked 40 in the world!
```

Big Data Tools and Algorithm Exploration

Applied Machine Learning Algorithm Selection

HKUSPACE

Big Data Tools and Algorithm Exploration
Applied Machine Learning Algorithm Selection



Inspiring Your Future

Business Education @ HKUSPACE

Source: <https://elitedatascience.com/algorithm-selection>

Big Data Tools and Algorithm Exploration

Applied Machine Learning Algorithm Selection

Each tree is only trained on a random subset of observations (a process called resampling)

Each tree is only allowed to choose from a random subset of features to split on (leading to feature selection)



Random Forests

Random forests train a large number of "strong" decision trees and combine their predictions through bagging

random forests tend to perform very well right out of the box often beat many other models that take up to weeks to develop, and are the perfect "*swiss-army-knife*" algorithm that almost always gets good results

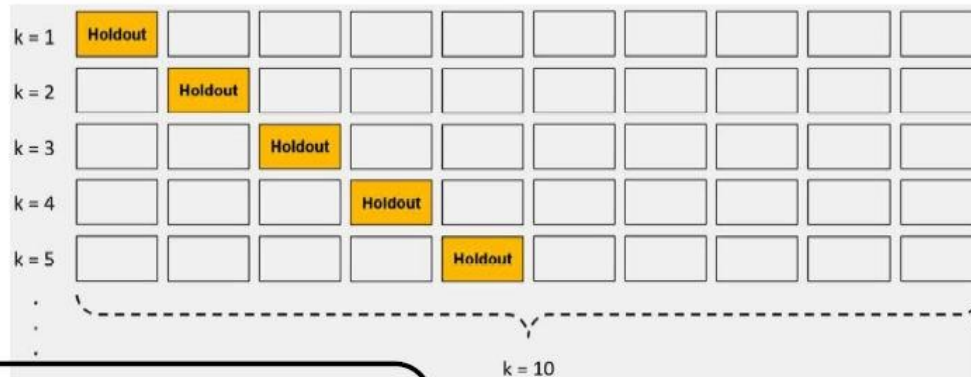
Source: <https://elitedatascience.com/algorithm-selection>

Big Data Tools and Algorithm Exploration

Model Training

1- Split your data into 10 equal parts, or “folds”, 2- Train your model on 9 folds (e.g. the first 9 folds)

Cross-validation is a tuning method for getting a reliable estimate of model performance using only training data



3 - Evaluate it on the 1 remaining "hold-out" fold, Perform steps (2) and (3) 10 times, each time holding out a different fold

10-fold cross-validation, breaks your training data into 10 equal parts (or *folds*), essentially creating 10 miniature train/test splits

Average the performance across all 10 hold-out folds, average performance across the 10 hold-out folds is your final performance estimate, also called your *cross-validated score*

Source: <https://elitedatascience.com/model-training>

Big Data Tools and Algorithm Exploration

Model Training

perform the entire cross-validation loop detailed above on each set of hyperparameter values using a high-level pseudo-code

We've split our dataset into training and test sets, and we've learned about hyperparameters and cross-validation, we're ready to fit and tune our models

For each algorithm (i.e. regularized regression, random forest, etc.):

For each set of hyperparameter values to try:

Perform cross-validation using the training set.

Calculate cross-validated score.

At the end of this process, you will have a cross-validated score for each set of hyperparameter values... for each algorithm

Elastic-Net		
Penalty Ratio	Penalty Strength	CV-Score
75/25	0.01	0.63
75/25	0.05	0.64
75/25	0.10	0.67
50/50	0.01	0.62
50/50	0.05	0.63
50/50	0.10	0.66

Source: <https://elitedatascience.com/model-training>

Big Data Tools and Algorithm Exploration

Model Training

Keep the set of hyperparameter values with best cross-validated score. Re-train the algorithm on the entire training set (without cross-validation)

By now, you'll have 1 "best" model *for each algorithm* that has been tuned through cross-validation. Most importantly, you've only used the training data so far

Because you've saved your test set as a truly unseen dataset, you can now use it to get a reliable estimate of each model's performance. There are a variety of performance metrics you could choose from

For classification tasks, we recommend Area Under ROC Curve (AUROC). *Higher values are better*

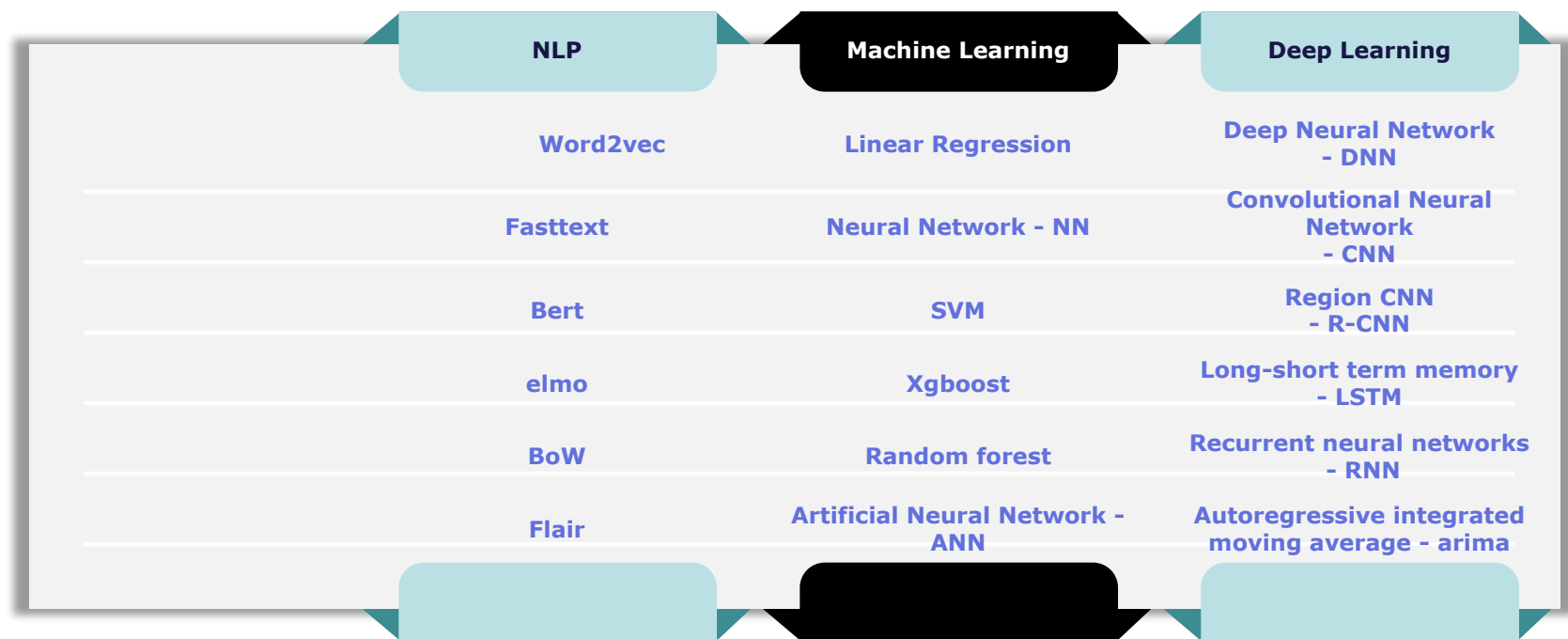
For regression tasks, we recommend Mean Squared Error (MSE) or Mean Absolute Error (MAE). (*Lower values are better*)



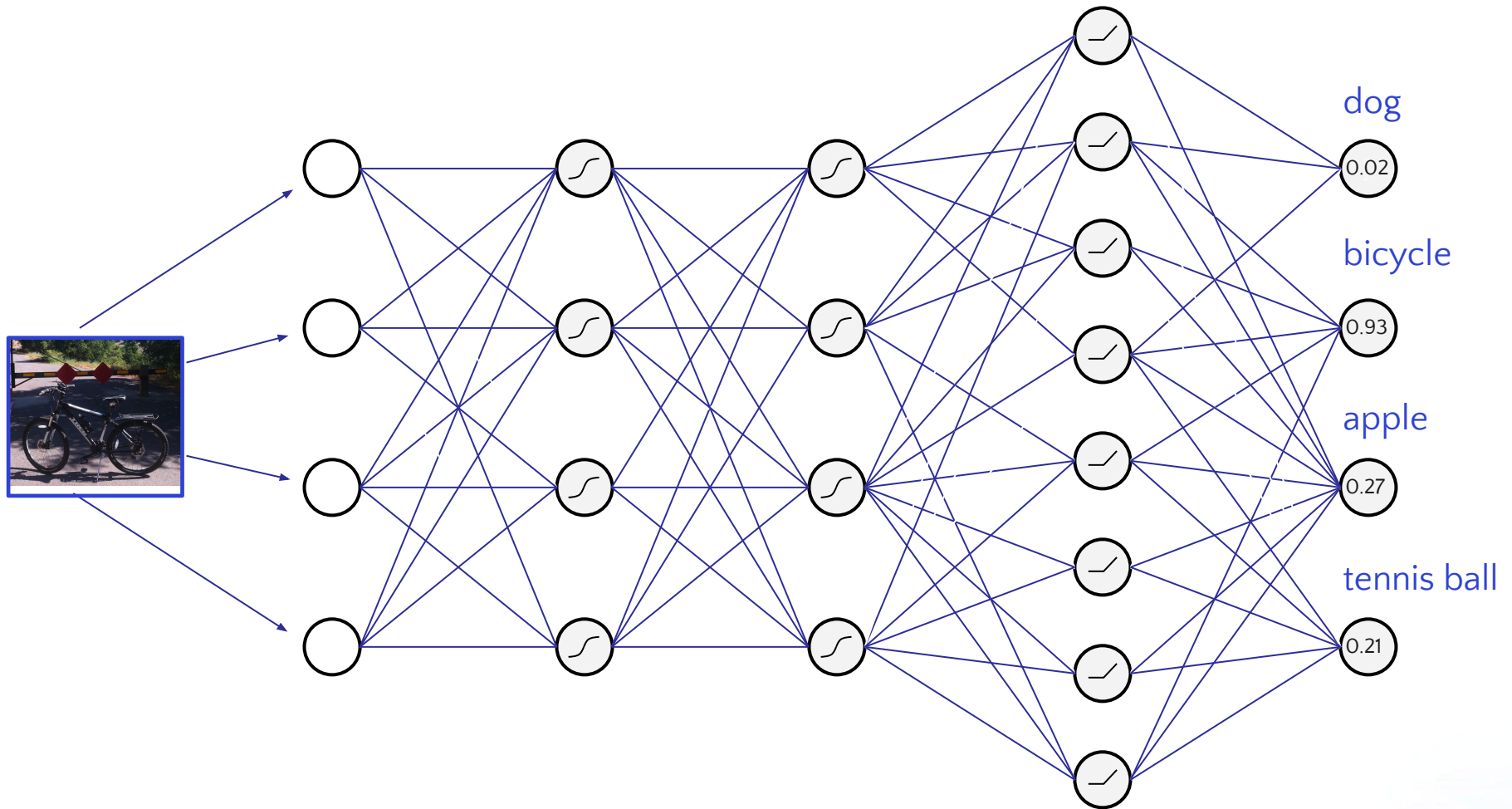
like the Hunger Games... each algorithm sends its own "representatives" (i.e. model trained on the best set of hyperparameter values) to the final selection

Source: <https://elitedatascience.com/model-training>

Modelling Techniques

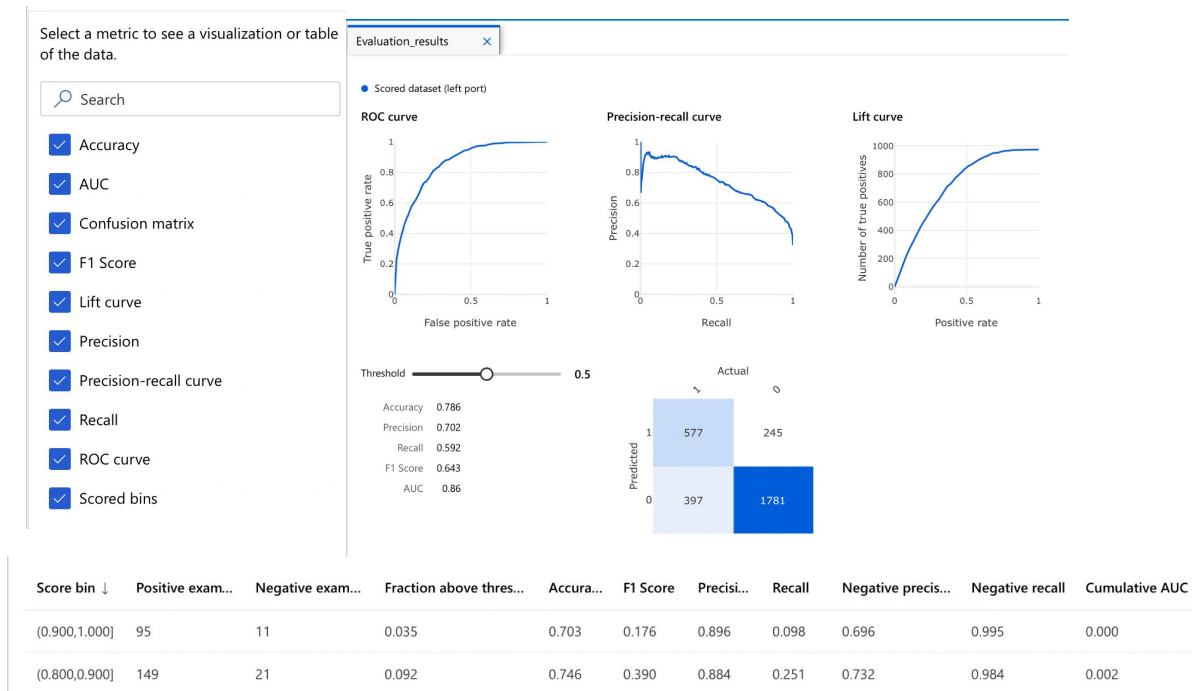


Trained Convolutional NN



Machine Learning Model Training Visualization

Machine learning model training visualization involves representing various aspects of the training process and the resulting model through graphical or interactive means. This aims to enhance understanding, facilitate debugging, and improve interpretability for both technical and non-technical stakeholders



<https://www.analyticsvidhya.com/blog/2021/09/a-comprehensive-guide-on-using-azure-machine-learning/>

Machine Learning Model Training Visualization

Key Concepts in Machine Learning Model Training Visualization:

- Monitoring Training Progress:
 - Loss Curves: Plots showing the training and validation loss over epochs or iterations, indicating convergence and potential overfitting/underfitting.
 - Interpretation: Both decreasing and converging, Training loss decreasing, validation loss increasing.
 - Metric Curves: Visualizations of performance metrics (e.g., accuracy, precision, recall, F1-score) on training and validation sets, providing insight into model effectiveness.
 - Interpretation: High and stable validation metric, Significant gap between training and validation metrics, Low metrics on both sets
 - Learning Rate Schedules: Graphs illustrating how the learning rate changes during training, especially with adaptive optimizers or learning rate schedulers
 - Interpretation: Visualizing the schedule, Impact on convergence, Tuning

Key Features of ML in Financial Forecasting

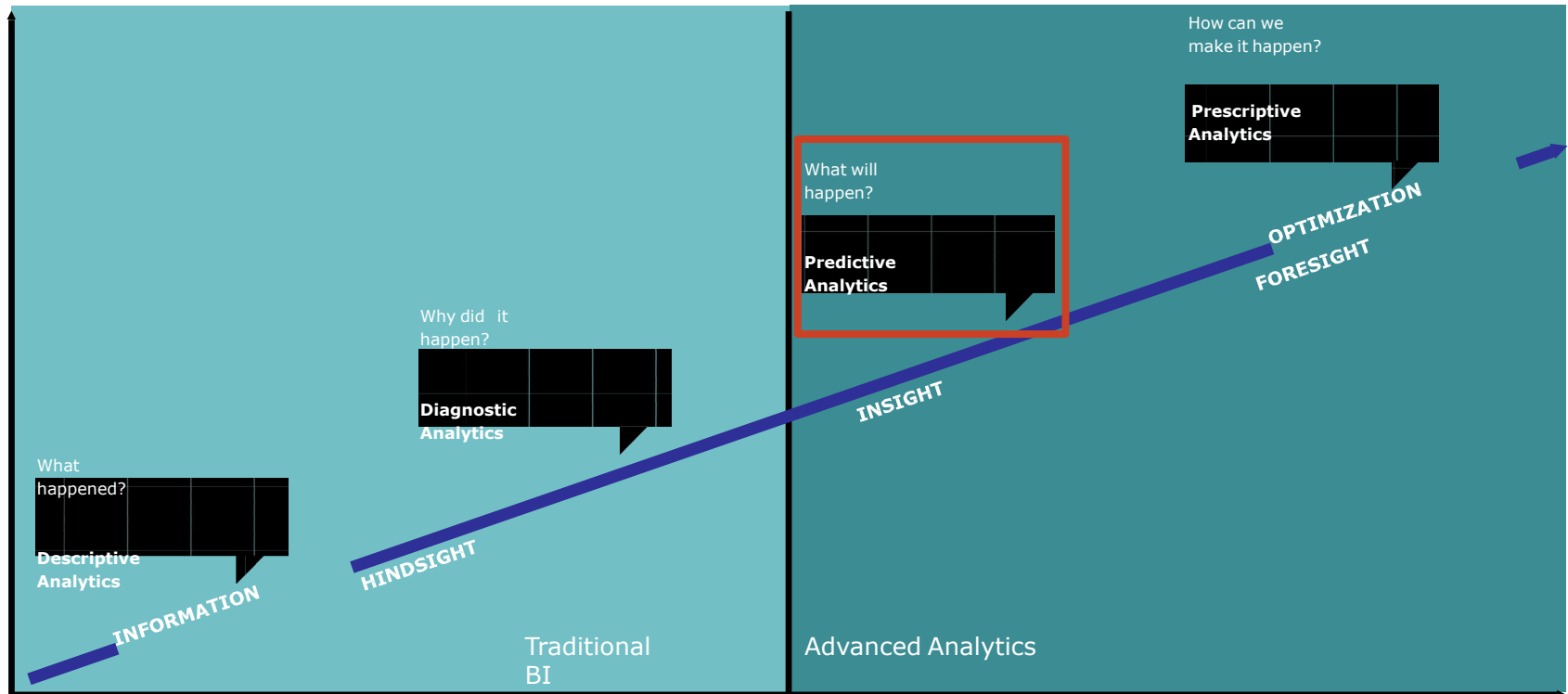
- Pattern recognition of non-linear relationships in prices, volumes, and macro indicators that traditional linear models miss
- Support for multiple forecast targets: prices, returns, volatility, default probability, credit risk, and cash-flow or revenue trajectories
- Integration of unstructured data (text, sometimes images) so that sentiment and qualitative information can influence quantitative forecasts (<https://www.netsuite.com.hk/portal/hk/resource/articles/financial-management/financial-forecast-machine-learning.shtml>)

Core Machine Learning Technologies Used

- Time-series models, including ARIMA and LSTM, to capture temporal dependencies in financial data
- Tree-based methods such as decision trees, random forests, and gradient boosting (XGBoost, LightGBM, CatBoost) for credit scoring, risk forecasting, and tabular financial data.

Tutorial - <https://medium.com/@aditib259/predicting-stock-prices-using-lstms-time-series-forecasting-a-step-by-step-guide-a70ebb04bbb8>

Financial Forecasting with Machine Learning using Python

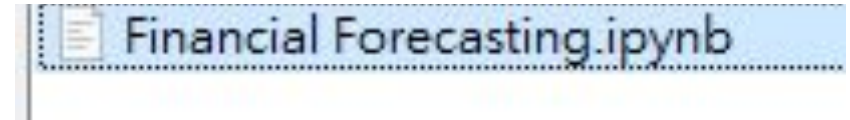


Extra Lab - Financial Forecasting with Machine Learning using Python (Numpy, Pandas, Matplotlib and Scikit-learn)

Open Google Colab

Step 1. Import the required libraries:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from keras.models import Sequential
from keras.layers import Dense, LSTM
import requests
```



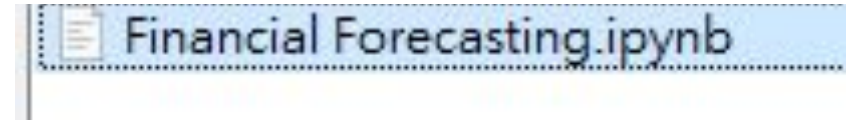
Step 2: Load the finance data from <https://www.alphavantage.co>:

```
api_key = 'demo'
symbol = 'IBM'
url =
f'https://www.alphavantage.co/query?function=TIME_SERIES_DAILY_ADJUSTED&symbol={symbol}&outputsize=full&a
pikey={api_key}'
response = requests.get(url)
data = response.json()
df = pd.DataFrame(data['Time Series (Daily)']).transpose()
df.index = pd.to_datetime(df.index)
df = df.sort_index()
```

Financial Forecasting with Machine Learning using Python (Numpy, Pandas, Matplotlib and Scikit-learn)

Step 3 – Preprocess the data:

```
# Extract the closing prices
y = df['4. close'].values.astype(float)
# Normalize the closing prices
scaler = MinMaxScaler(feature_range=(0, 1))
y = scaler.fit_transform(y.reshape(-1, 1))
# Create the feature matrix
X = []
for i in range(60, len(df)):
    X.append(y[i-60:i, 0])
X = np.array(X) # Split the data into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X, y[60:], test_size=0.2, shuffle=False)
```



Step 4 – Define the model:

```
model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(LSTM(units=50))
model.add(Dense(units=1))
model.compile(optimizer='adam', loss='mean_squared_error')
```

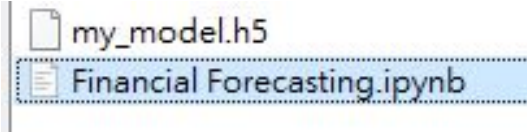
Financial Forecasting with Machine Learning using Python (Numpy, Pandas, Matplotlib and Scikit-learn)

Step 5 – Train the model with 100 steps:

```
model.fit(X_train, y_train, epochs=100, batch_size=32, validation_data=(X_val, y_val))
```

Step 6 – Save the model in h5 format:

```
model.save('my_model.h5') # Save the model in HDF5 format
```



Step 7 – Evaluate the model:

```
# Evaluate the model on the validation set
```

```
y_pred = model.predict(X_val)
```

```
rmse = np.sqrt(mean_squared_error(y_val, y_pred))
```

```
print('Root Mean Squared Error:', rmse)
```

Step 8 – Visualize the results:

```
# Visualize the results
```

```
y_pred = scaler.inverse_transform(y_pred)
```

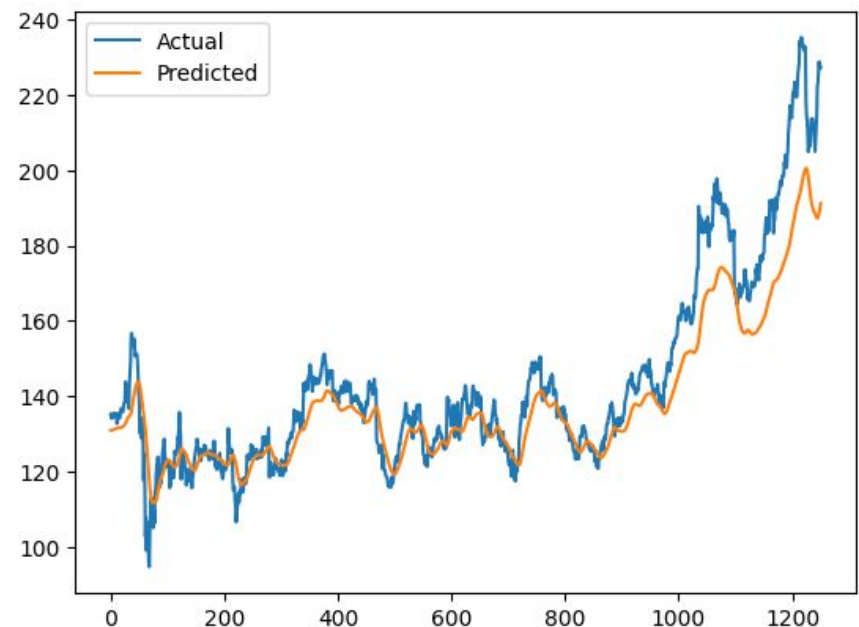
```
y_val = scaler.inverse_transform(y_val)
```

```
plt.plot(y_val, label='Actual')
```

```
plt.plot(y_pred, label='Predicted')
```

```
plt.legend()
```

```
plt.show()
```



Extra Lab - Google Sheet Content Update

1. Open Anaconda Prompt, enter "pip install gspread google-auth google-api-python-client" to install google library
2. Open Jupyter notebook and type following python code

```
import gspread
from google.oauth2.service_account import Credentials
from googleapiclient.discovery import build

def update_values(sheet_id, range_name, value_input_option, values):
    SCOPES = ['https://www.googleapis.com/auth/spreadsheets']
    SERVICES_ACCOUNT_FILE = "graceful-disk-484104-a2-152ebd9c692e.json"

    credentials = Credentials.from_service_account_file(SERVICES_ACCOUNT_FILE,
                                                       scopes=SCOPES)
    service = build('sheets', 'v4', credentials=credentials)

    body = {"values": values}
```

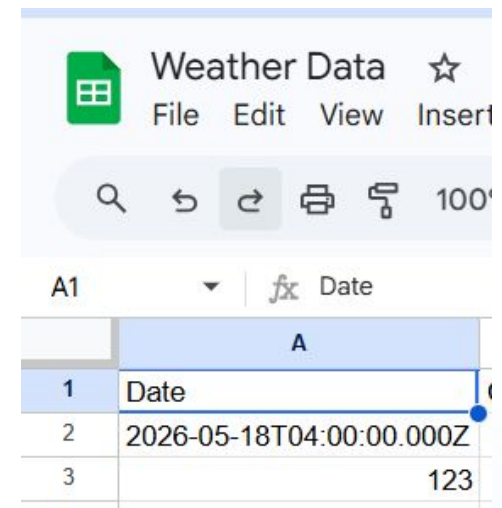
Google Sheet Content Update

```
# Update the range to just the starting cell for appending
result = service.spreadsheets().values().update(
    spreadsheetId=sheet_id,
    range=range_name, # Change range here
    valueInputOption=value_input_option,
    body=body
).execute()
```

```
print(f"{result.get('updatedCells')} cells updated.")
return result
```

```
sheet_id = '1mqrumbU1Si5aGmkg2jpffu7zn9uWxFvXX1Wnuwkas'
range_name = "Sheet1!A3"
value_input_option = 'USER_ENTERED'
values = [['123']]
```

```
update_values(sheet_id, range_name, value_input_option, values)
```



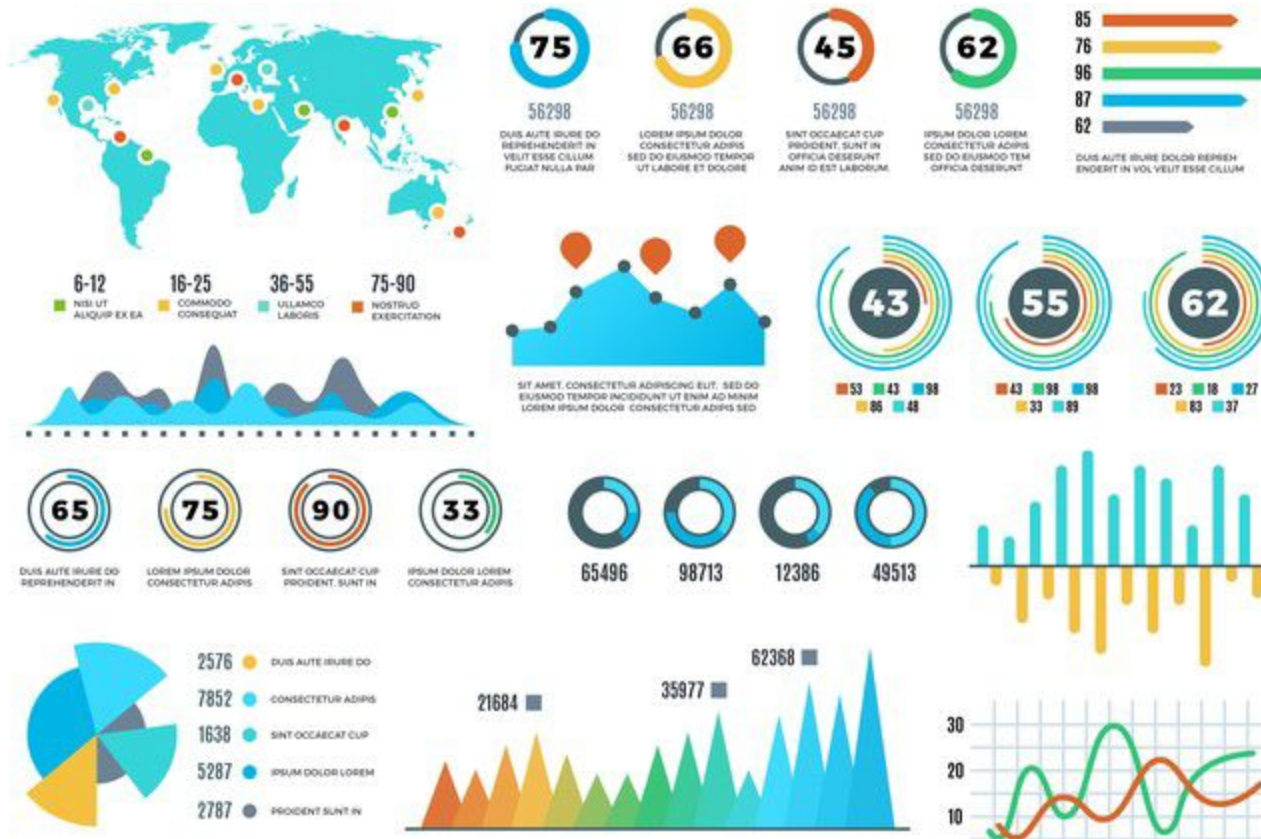
Weather Data	
File	Edit View Insert
A1 fx Date	
1	Date
2	2026-05-18T04:00:00.000Z
3	123



Data Visualizations

1. describe the contemporary trends of process automation and explain the opportunities and challenges of business process automation;
2. outline key steps in process automation project management and illustrate the significance of each step;
3. apply computational tools to implement process automation;
4. discuss the development of process automation and practical cases for business.

Data Visualizations



Data Visualizations

- Data visualization is the practice of translating information into a visual context, such as a map or graph, to make data easier for the human brain to understand and pull insights from.
- The main goal of data visualization is to make it easier to identify patterns, trends and outliers in large data sets.
- The term is often used interchangeably with others, including information graphics, information visualization and statistical graphics.
- Data visualization is one of the steps of the data science process, which states that after data has been collected, processed and modeled, it must be visualized for conclusions to be made.
- Data visualization is also an element of the broader data presentation architecture (DPA) discipline, which aims to identify, locate, manipulate, format and deliver data in the most efficient way possible.



Benefits of Data Visualizations

- The ability to absorb information quickly, improve insights and make faster decisions;
- An increased understanding of the next steps that must be taken to improve the organization;
- An improved ability to maintain the audience's interest with information they can understand;
- An easy distribution of information that increases the opportunity to share insights with everyone involved;
- Eliminate the need for data scientists since data is more accessible and understandable; and
- An increased ability to act on findings quickly and, therefore, achieve success with greater speed and less mistakes.

Data Visualization Roles - Change over time



Line chart



+Comparisons

Most common chart type for showing change over time. A point is plotted for each time period from left to right; each point's vertical position indicates the feature's value. Points are connected by line segments to emphasize progression across time.



Sparkline



+Comparisons

A miniature line chart with little to no labeling, designed to be placed alongside text or in tables. Provides a high-level overview without attracting too much attention. Can also be seen in a sparkbar form, or miniature bar chart (see below).

Data Visualization Roles - Change over time



Connected scatter plot



+Relationships

Shows change over time across two numeric variables (see scatter plot in *Relationships*). Line segments still connect points across time, but they may not consistently go from left to right like in a line chart.



Bar chart

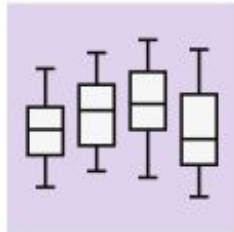


+Distributions

+Comparisons

Each time period is associated with a bar; each bar's value is represented in its height above (or below) a zero-baseline. Works best when there aren't too many time periods to show.

Data Visualization Roles - Change over time



Box plot

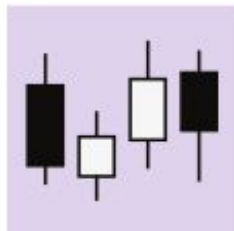


+Distributions

+Comparisons

Each time period is associated with a box and whiskers; each set of box and whiskers shows the range of the most common data values. Best when there are multiple recordings for each time period and a distribution of values needs to be plotted.

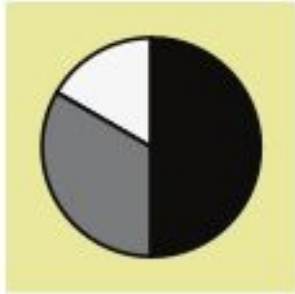
Tracking change over time is of key interest in the financial domain. One specialist chart developed for this field includes the following:



Candlestick chart ◆

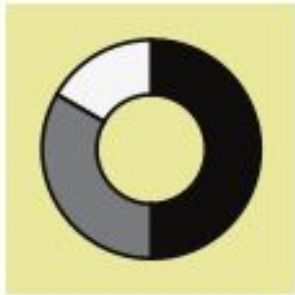
Looks like a box plot, but each box and whiskers encodes different statistics. The box ends indicate opening and closing prices, while color indicates the direction of change.

Data Visualization Roles - Part-to-whole composition



Pie chart ●

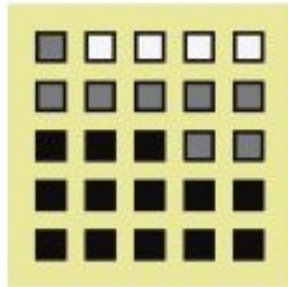
The whole is represented by a filled circle. Parts are proportional slices from that circle, one for each categorical group. Best with five or fewer slices with distinct proportions.



Doughnut chart ●

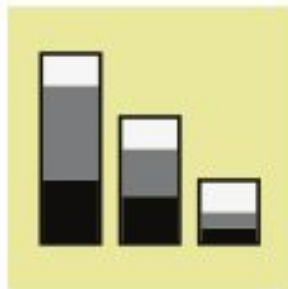
A pie chart with a hole in the center. This central area can be used to show a relevant single numeric value. Sometimes used as an aesthetic alternative to a standard progress bar (see stacked bar chart below).

Data Visualization Roles - Part-to-whole composition



Waffle chart / grid plot

Squares laid out in a (typically) 10 x 10 grid; each square represents one percent of the whole. Squares are colored based on categorical group size.



Stacked bar chart

A bar chart (see *Change over time* or *Distributions*) where each bar has been divided into multiple sub-bars to show a part-to-whole breakdown. A single stacked bar can be used as an alternative to the pie or doughnut chart; people tend to make more precise judgments of length over area or angle.

Data Visualization Roles - Part-to-whole composition



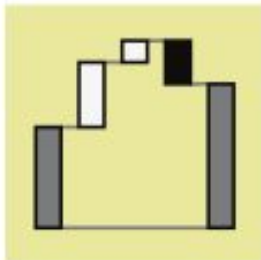
Stacked area chart ●

A line chart (see *Change over time*) where shaded regions are added under the line to divide the total into sub-group values.



Stream graph ◆

Modified version of the stacked area chart where areas are stacked around a central axis. Highlights relative changes instead of exact values.

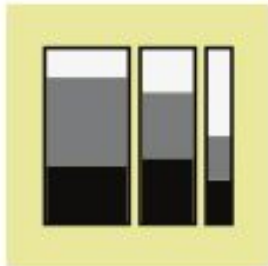


Waterfall chart ◆

Augments a change over time with a part-to-whole decomposition. Bars on the ends depict values at two time points, and lengths of intermediate floating bars' show the decomposition of the change between points.

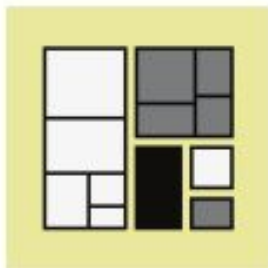
Data Visualization Roles - Part-to-whole composition

Certain part-to-whole compositions follow a hierarchical form. In these cases, each part can be divided into finer parts on lower levels. Here are a couple of more specialized chart types for visualizing this type of data:



Mosaic plot / Marimekko chart ■

Can be thought of as a stacked bar divided on both axes. A box is divided on one axis based on one categorical variable, then each sub-box is divided in the other axis based on a second categorical variable.



Treemap ◆

Can be thought of as a more generalized Marimekko plot. Sub-boxes do not need to have a consistent cut direction at a particular hierarchy level, and there can be more than two levels of hierarchy.

Data Visualization Roles - Flows and processes



Funnel chart

Seen in business contexts, showing how people encounter a product and eventually become users or customers. One bar is plotted for each stage, whose lengths reflect the number of users. Connecting regions emphasize connections in stages and give the chart type's namesake shape.



Parallel sets chart

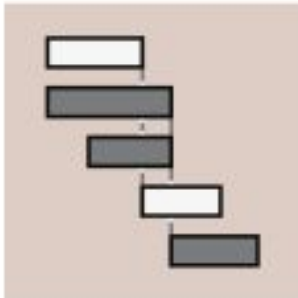
Multiple part-to-whole divisions on different dimensions are depicted as parallel stacked bars. Connecting regions show how different subgroups relate to one another between dimensions.

Data Visualization Roles - Flows and processes



Sankey diagram ◆

The width of the colored region shows the relative volume at each part of a process. Allows for multiple sources of inputs and outputs to be visualized.



Gantt chart ■

Used for project scheduling, breaking them down into individual tasks. Each task is associated with a bar, providing a timeline for when each task should begin and end.

Data Visualization Roles - Geographical data



Scatter map ●

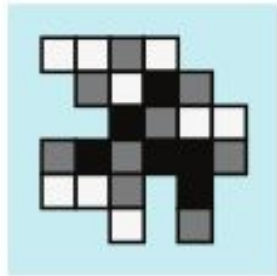
Scatter plot built on top of a geographical map, using geographic coordinates as point positions.



Bubble map ●

Bubble chart built on top of a geographic map, where point size is an indicator of value. Can also be used to group together points in a scatter map if they are too dense.

Data Visualization Roles - Geographical data



2-d histogram ●

Heatmaps can be built on top of geographic areas. Sometimes seen with a hexagon-shaped grid rather than a rectangular grid. May distort the geography on its edges.



Isopleth / contour map ◆

2-d density curve built on top of a geographic map.



Connection map ◆

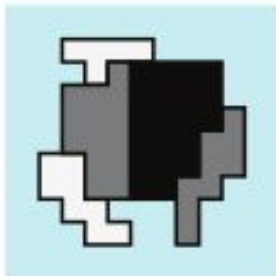
Network information and flows built on top of a geographic map.

Data Visualization Roles - Geographical data



Choropleth ●

Similar to a heatmap, but colors are assigned to geopolitical regions rather than an arbitrary grid. Values are often in the form of rates or ratios to avoid distortion due to population density.



Cartogram ◆

Geopolitical regions sized by value. This necessarily requires distortion in shapes and topology.

Visualization using Programming

- Python
 - **matplotlib**
 - seaborn
 - **plotly(mainly)**
 - pylab
- R
 - graphics
 - ggplot2

Plotly: Visualization with Python

Plotly's Python graphing library makes interactive, publication-quality graphs.

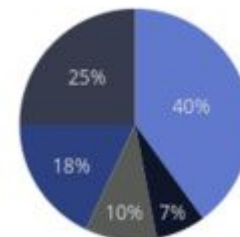
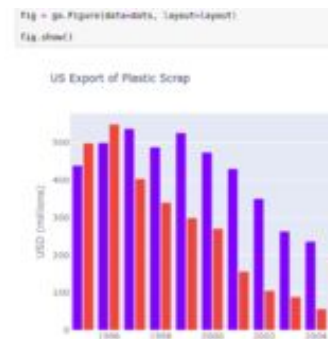
Examples of how to make line plots, scatter plots, area charts, bar charts, error bars, box plots, histograms, heatmaps, subplots, multiple-axes, polar charts, and bubble charts.

Plotly.py is free and open source..

Basic Chart: <https://plotly.com/python/basic-charts/>

Features:

1. Create publication quality plots.
2. Make interactive figures that can zoom, pan, update.
3. Customize visual style and layout.
4. Export to many file formats.
5. Embed in JupyterLab and Graphical User Interfaces.
6. Use a rich array of third-party packages built on Matplotlib.



<https://plotly.com/python/>

Plotly: Visualization with Python

The main modules in Plotly are:

- `plotly.plotly`
 - **`plotly.graph.objects`**
 - `plotly.express`
-
- `plotly.plotly` acts as the interface between the local machine and Plotly. It contains functions that require a response from Plotly's server.
 - **`plotly.graph_objects` module contains the objects (Figure, layout, data, and the definition of the plots like scatter plot, line chart) that are responsible for creating the plots.**
 - `plotly.express` module can create the entire Figure at once. It uses the `graph_objects` internally and returns the `graph_objects.Figure` instance.

Plotly Exercise 1

```
#using plotly, display histogram chart and download chart
import numpy as np
import plotly.graph_objects as go
```

```
# Generate sample data for the sine wave
x = np.linspace(0, 10, 100) # 100 points from 0 to 10
y = np.sin(x)                # y is the sine of x
```

```
# Generate sample data for the histogram
data = np.random.normal(loc=0, scale=1, size=1000)
random values from a normal distribution
```

```
# Create a plotly figure
fig = go.Figure()
```

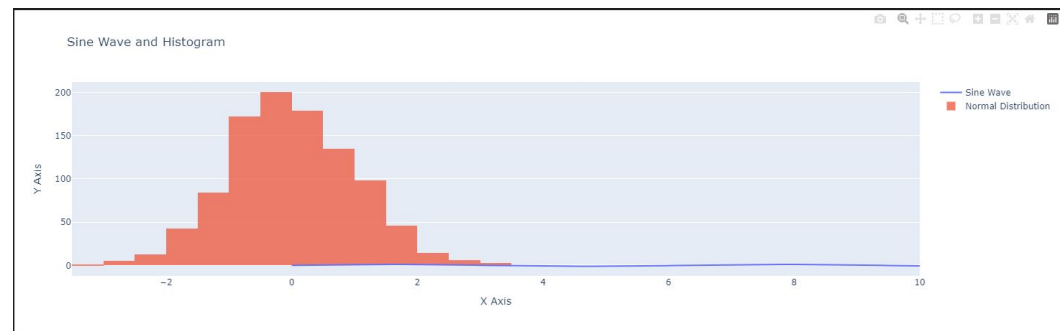
```
# Add a line plot for the sine wave
fig.add_trace(go.Scatter(x=x, y=y, mode='lines',
                        name='Sine Wave'))
```

```
# Add a histogram
fig.add_trace(go.Histogram(x=data, name='Normal
Distribution', opacity=0.75, nbinsx=30))
```

```
# Customize layout
fig.update_layout(title='Sine Wave and Histogram',
                  xaxis_title='X Axis',
                  yaxis_title='Y Axis',
                  bargmode='overlay') # Overlay histograms
```

```
# Show the plot
fig.show()
# Show the plot
fig.show()
```

```
# Save the figure as a PNG filep
io.write_image(fig, 'sine_wave_histogram.png')
```



Plotly Example - Line Chart

```
import plotly.graph_objects as go
```

```
# Create random data with numpy
```

```
import numpy as np
```

```
np.random.seed(1)
```

```
N = 100
```

```
random_x = np.linspace(0, 1, N)
```

```
random_y0 = np.random.randn(N) + 5
```

```
random_y1 = np.random.randn(N)
```

```
random_y2 = np.random.randn(N) - 5
```

```
# Create traces
```

```
fig = go.Figure()
```

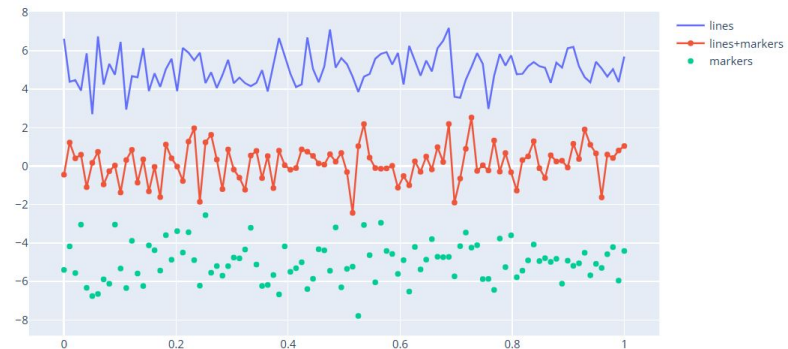
```
fig.add_trace(go.Scatter(x=random_x, y=random_y0,  
                        mode='lines',  
                        name='lines'))
```

```
fig.add_trace(go.Scatter(x=random_x, y=random_y1,  
                        mode='lines+markers',  
                        name='lines+markers'))
```

```
fig.add_trace(go.Scatter(x=random_x, y=random_y2,  
                        mode='markers', name='markers'))
```

```
fig.show()
```

<https://plotly.com/python/line-charts/>



Plotly Example - Bar Chart

```
import plotly.graph_objects as go
```

<https://plotly.com/python/bar-charts/>

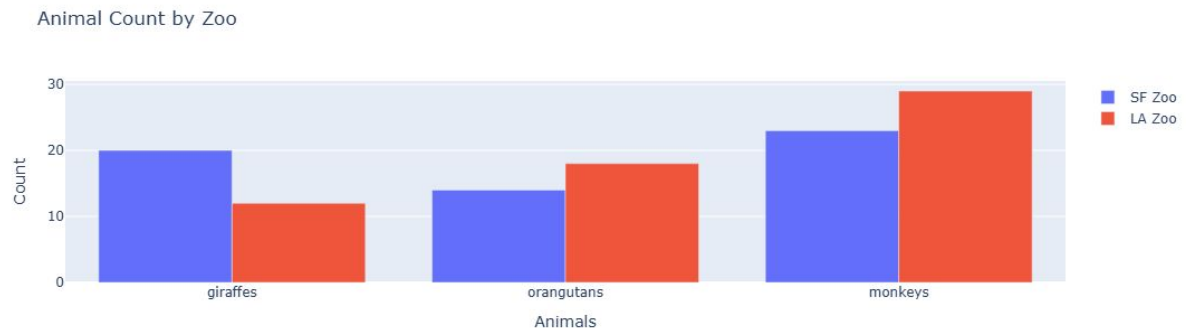
```
animals = ['giraffes', 'orangutans', 'monkeys']
```

```
fig = go.Figure(data=[  
    go.Bar(name='SF Zoo', x=animals, y=[20, 14, 23]),  
    go.Bar(name='LA Zoo', x=animals, y=[12, 18, 29])  
])
```

```
# Change the bar mode
```

```
fig.update_layout(  
    barmode='group',  
    title='Animal Count by Zoo',  
    xaxis_title='Animals',  
    yaxis_title='Count'  
)
```

```
fig.show()
```



Plotly Example - Scatter Chart

```
import plotly.graph_objects as go
import pandas as pd
```

<https://plotly.com/python/line-and-scatter/>

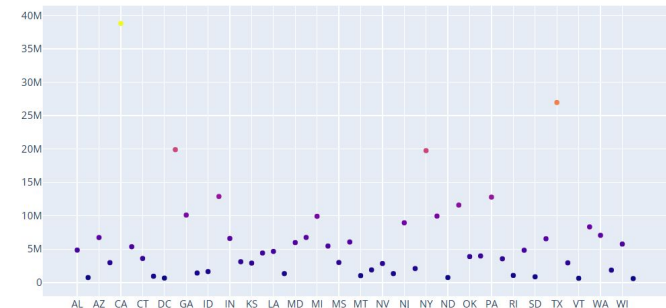
```
data= pd.read_csv("https://raw.githubusercontent.com/plotly/datasets/master/2014_usa_states.csv")
```

```
fig = go.Figure(data=go.Scatter(x=data['Postal'],
                                y=data['Population'],
                                mode='markers',
                                marker_color=data['Population'],
                                text=data['State'])) # hover text goes here
```

```
fig.update_layout(title=dict(text='Population of USA States'))
```

```
fig.show()
```

Population of USA States



Plotly Example - Pie Chart

<https://plotly.com/python/pie-charts/>

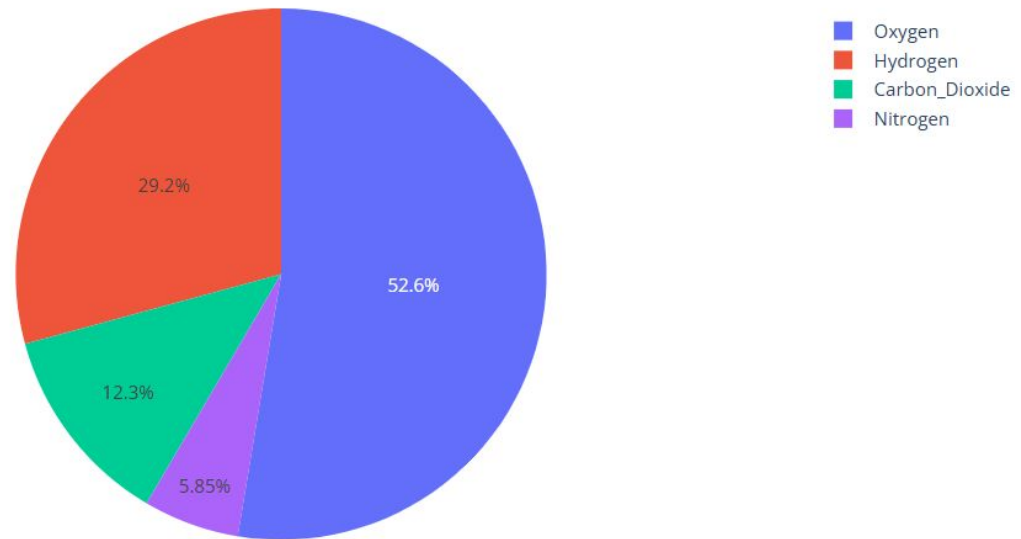
```
import plotly.graph_objects as go
```

```
labels = ['Oxygen','Hydrogen','Carbon_Dioxide','Nitrogen']
```

```
values = [4500, 2500, 1053, 500]
```

```
fig = go.Figure(data=[go.Pie(labels=labels, values=values)])
```

```
fig.show()
```



Plotly Example - Gantt Chart

<https://plotly.com/python/gantt/>

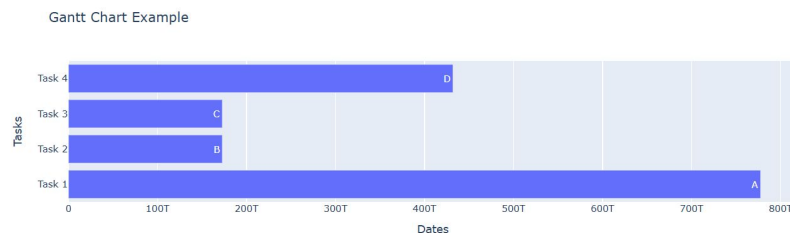
```
Import plotly.graph_objects as go
import pandas as pd
```

```
# Sample data for Gantt Chart
```

```
data = {
    'Task': ['Task 1', 'Task 2', 'Task 3', 'Task 4'],
    'Start': ['2023-01-01', '2023-01-13', '2023-01-18', '2023-01-25'],
    'Finish': ['2023-01-10', '2023-01-15', '2023-01-20', '2023-01-30'],
    'Resource': ['A', 'B', 'C', 'D']
}
```

```
# Convert to DataFrame
```

```
df = pd.DataFrame(data)
```



```
# Create Gantt chart
```

```
fig = go.Figure(data=[
    go.Bar(
        y=df['Task'],
        x=pd.to_datetime(df['Finish']) - pd.to_datetime(df['Start']),
        base=pd.to_datetime(df['Start']),
        orientation='h',
        text=df['Resource'],
        hoverinfo='text'
    )
])
```

```
# Update layout
```

```
fig.update_layout(
    title='Gantt Chart Example',
    yaxis_title='Tasks',
    xaxis_title='Dates',
    barmode='stack',
)
```

```
# Show the figure
```

```
fig.show()
```

Introduction of Python Framework

Flask - Flask is a **microframework** for Python. The main purpose is to develop a strong web application base. As compared to Django, Flask is best suited for **small and easy projects**.

Django - Django is a **high-level Python** Web application development framework that encourages us to develop things rapidly. It's free and open source.

Web2Py - It is a free open source full-stack development framework in python which allows the user to develop things quickly. It is a **cross-platform framework** that supports all popular operating systems.



Commonly used Flash Framework

Key features of Django



Authentication



Security



Object-relational mapping



Templates



Forms

django

Comparison between Django & Flask

	 django	 Flask
Type of Framework	Full Stack Web Framework	WSGI framework
Flexibility	Feature-packed	Full flexibility
ORM Usage	Built-in ORM	SQLAlchemy is used
Design	Batteries-included	Minimalistic design
Working Style	Monolithic	Diversified

What is Dash / Python Flask Framework?

- **Dash** is **Python framework** for building web applications. It is open source, and its app run on the web browser
- It built on top of **Flask**, Plotly.js, React and React Js.
- It enables you to build dashboards using pure **Python**.

What is Dash / Python Flask Framework?

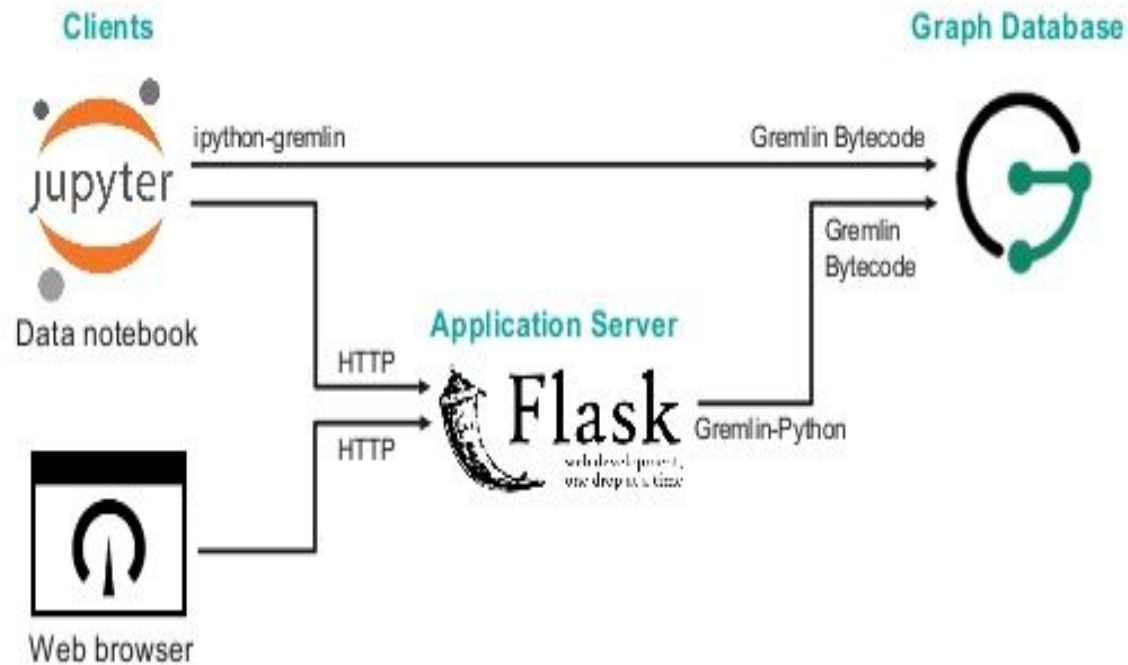
Every Dash App is composed of two main parts:

Layout = is used to define how all the content should be laid out on the application.

Callbacks = are used to make each of the desired components of the Dashboard interactive.

Python Flask Framework Architecture

Example Python Application Architecture



What is a Project?

“Unique process consisting of a set of coordinated and controlled **activities** with **start** and **finish** dates, undertaken to achieve an objective conforming to specific requirements, including constraints of **time**, **cost**, **quality** and **resources**”

A Project is a planned set of activities

A Project has a scope

A Project has time, cost, quality and resource constraints

What is Project Management?

The art of organising, leading, reporting and completing a project through people

- A project is a planned undertaking
- A project manager is a person who causes things to happen

Therefore, project management is causing a planned undertaking to happen.

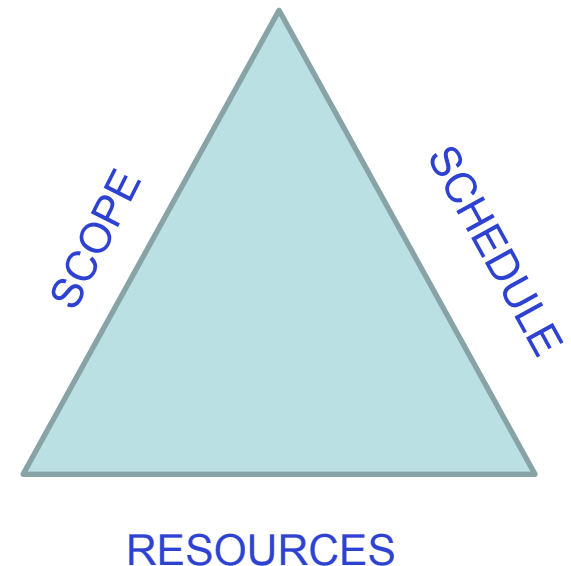


Project Management The “Triple Constraint”

The Project Manager / the researcher has certain constraints to battle with in delivering a project. These are referred to as “The Triple Constraint”. They include:

- Scope Management
- Cost Management
- Time Management
- Quality Assurance
- Human Resource Management
- Communication Management
- Risk Management
- Conflict Management among others.

PROJECT CONSTRAINT TRIANGLE



A Good Project Manager

Takes ownership of the whole project

Is proactive not reactive

Adequately plans the project

Is Authoritative (**NOT** Authoritarian)

Is Decisive

Is a Good Communicator

Manages by data and facts not uniformed optimism

Leads by example

Has sound Judgement

Is a Motivator

Is Diplomatic

Can Delegate





Stakeholder Engagement process

Identify Stakeholders

Assess needs

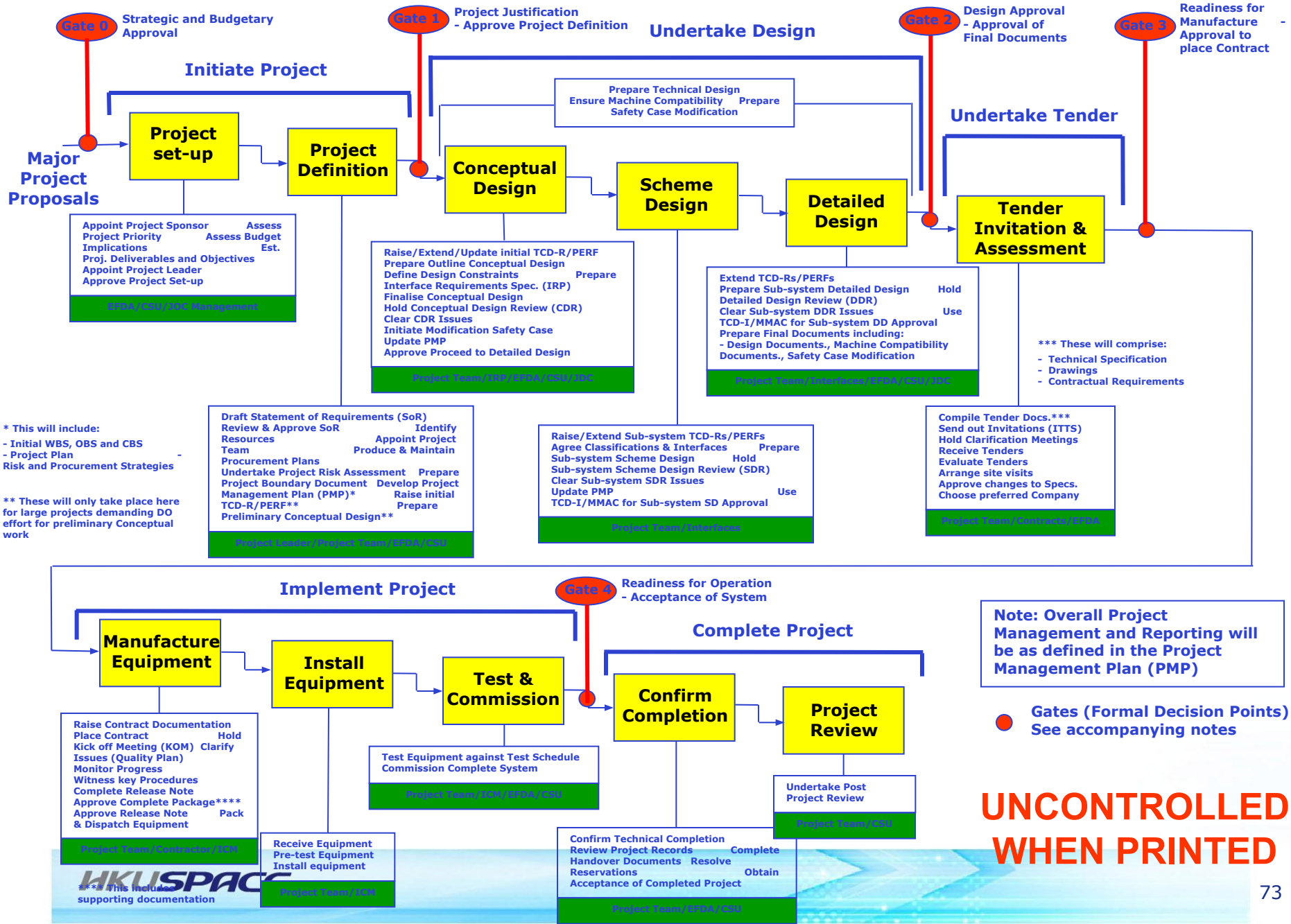
Define actions

Establish communication channels

Gather feedback

Monitor and review

The Project Process



Key Points in Project Set-up and Definition

- Create Project Management Plan (PMP)
- Be clear of scope and objectives
- Establish clear statement of what is to be done (WBS)
- Establish Risks to be Managed
- Establish Costs and Durations
- Establish Resources Required

Project management Plan - PMP

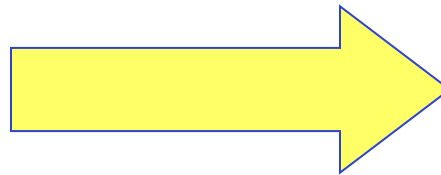
- Master Document for Project
- Defines the following:-
 - ⇒ Project Objectives, Scope, Deliverables
 - ⇒ Stakeholders (Internal & External)
 - ⇒ Work to be done (WBS)
 - ⇒ Project Organisation and Resources (OBS)
 - ⇒ Project Costings (CBS)
 - ⇒ Project Schedule
 - ⇒ Procurement/Contract Strategy
 - ⇒ Risk Management
 - ⇒ Quality management
 - ⇒ Change Management

Project Planning



Project Planning

Adequate planning leads to the correct completion of work

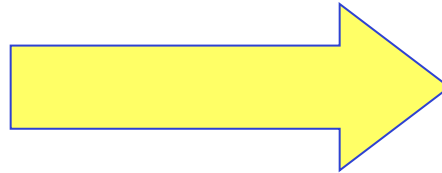


Plan

Inadequate planning leads to frustration towards the end of the project & poor project performance



Project Start



Project End

Work Breakdown Structure (WBS)

The Work Breakdown Structure is the foundation for effective project planning, costing and management.

It is the most important aspect in setting-up a Project

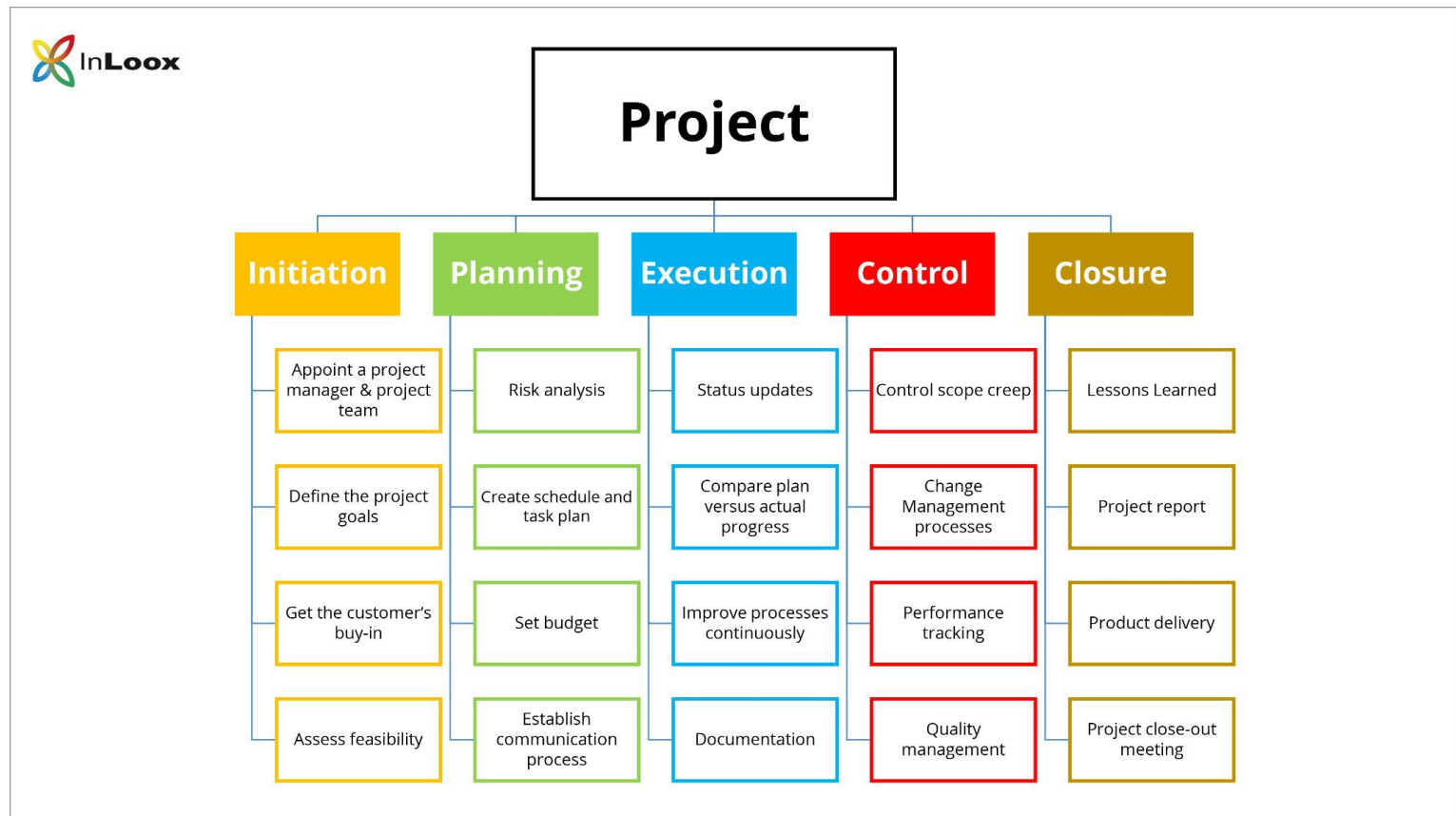
- It is the foundation on which everything else builds



Work Breakdown Structure - Definition

“A Work Breakdown Structure (WBS) is a hierarchical (from general to specific) tree structure of deliverables and tasks that need to be performed to complete a project.”

Example WBS - Top Level ILW Project



Example WBS - Top Level TSCL Project

Work breakdown structure levels

Level 1	Project goal or final deliverable								
Level 2	1. Major phase or deliverable			2. Major phase or deliverable			3. Major phase or deliverable		
Level 3	1.1 Sub-phase or deliverable	1.2 Sub-phase or deliverable	1.3 Sub-phase or deliverable	2.1 Sub-phase or deliverable	2.2 Sub-phase or deliverable	2.3 Sub-phase or deliverable	3.1 Sub-phase or deliverable	3.2 Sub-phase or deliverable	3.3 Sub-phase or deliverable
Level 4	1.1.1 Work package	1.2.1 Work package	1.3.1 Work package	2.1.1 Work package	2.2.1 Work package	2.3.1 Work package	3.1.1 Work package	3.2.1 Work package	3.3.1 Work package
	1.1.2 Work package	1.2.2 Work package	1.3.2 Work package	2.1.2 Work package	2.2.2 Work package	2.3.2 Work package	3.1.2 Work package	3.2.2 Work package	3.3.2 Work package
	1.1.3 Work package	1.2.3 Work package	1.3.3 Work package	2.1.3 Work package	2.2.3 Work package	2.3.3 Work package	3.1.3 Work package	3.2.3 Work package	3.3.3 Work package
	1.1.4 Work package		1.3.4 Work package	2.1.4 Work package	2.2.4 Work package	2.3.4 Work package	3.1.4 Work package	3.2.4 Work package	3.3.4 Work package
					2.2.5 Work package			3.2.5 Work package	3.3.5 Work package
									3.3.6 Work package

Tools for doing Project Management Plan

1. Gantt Charts: visualize timelines, track progress, and manage tasks effectively.
2. Asana: Offers flexible task management with various visualization options like lists, Kanban boards, and Gantt charts
3. Trello: A visual tool that organizes projects into boards and cards, ideal for agile workflows
4. Notion: Maintaining an internal database, collaborating, and managing tasks such as Creating project plans, Building roadmaps, Creating and storing important documents, guides, etc.
5. Monday.com : monday.com is a highly visual and intuitive project management tool that offers a range of customization options.
6. Jira : Jira is amongst the most established, powerful, and feature-complete project management tools available. It is straightforward to set up and available both on the cloud and on premises.
7. Wrike: Features task assignment, collaboration tools, and multiple viewing formats (list, board, Gantt) to manage workflows efficiently

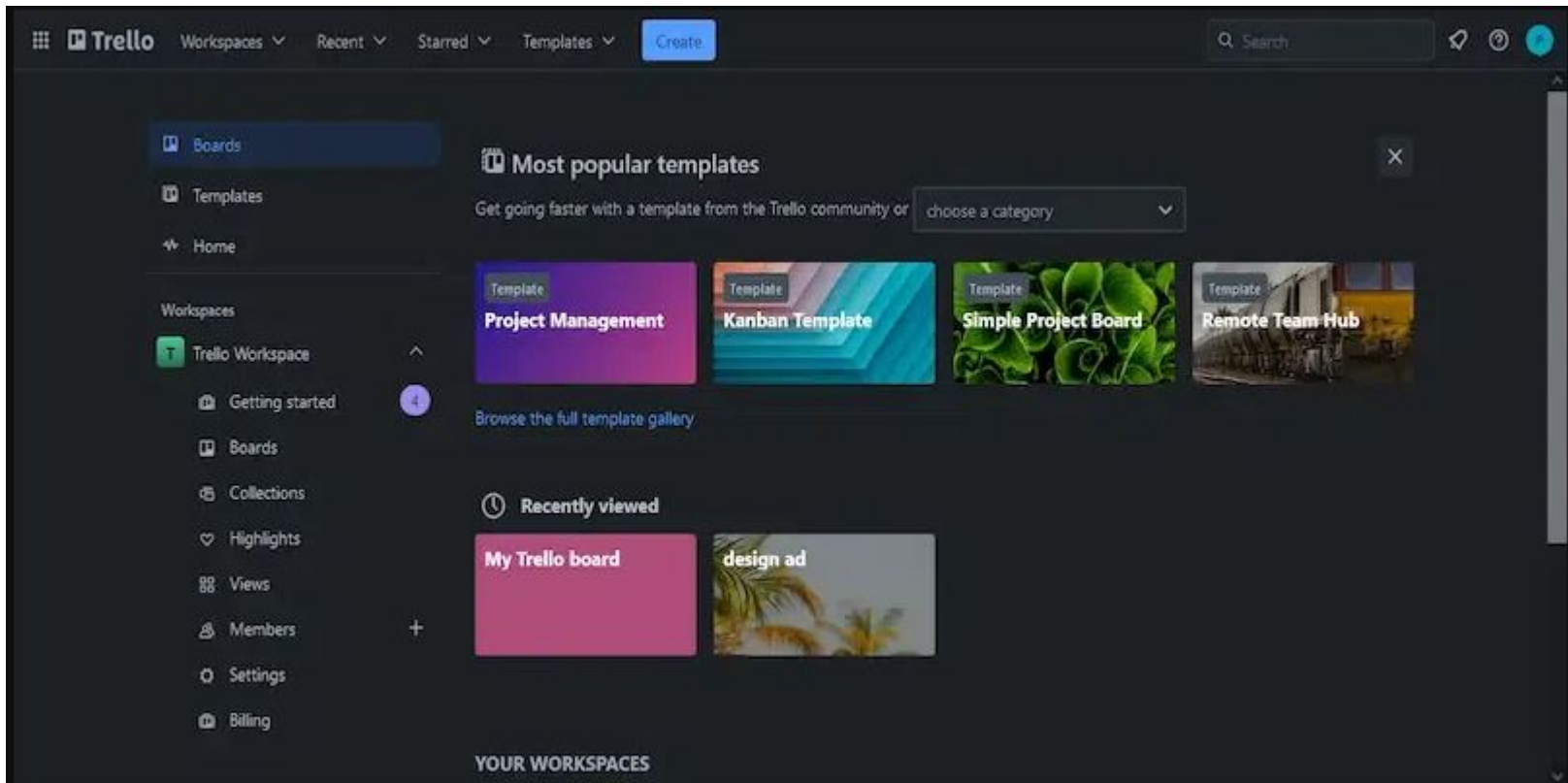


10 Ways to Use Trello for Project Management

1. Use Trello boards
2. Using Trello Cards
3. Calendar Planning
4. Email Replacement
5. Productivity Metrics
6. Automate Repetitive Email Tasks
7. CRM Tool
8. Arranging Meetings
9. Project Chat Channels
10. Time Tracking

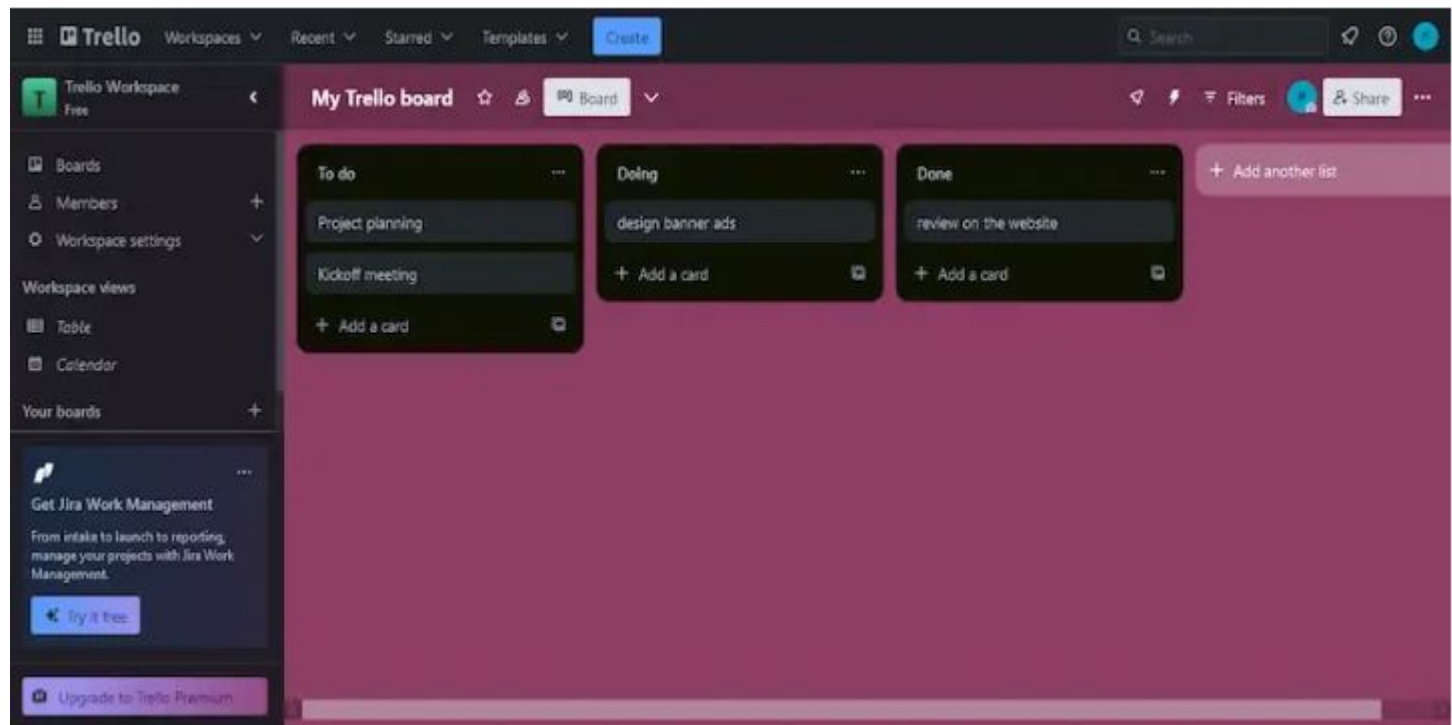
What is Trello?

Trello is a task and project management tool that's comprised of Boards, Lists, and Cards.



Trello Board

Trello boards are the advanced version of bulletin boards that are used to organize thoughts. The specialty of Trello is that you are not restricted to using a single board and can use multiple boards as complex projects will require multiple boards. It also provides different privacy options: **personal**, **private**, **team**, **organization**, and **public visibility**. Your board will show you **what is planned to be done**, and **task statuses**, and help you indicate capacity limitations.

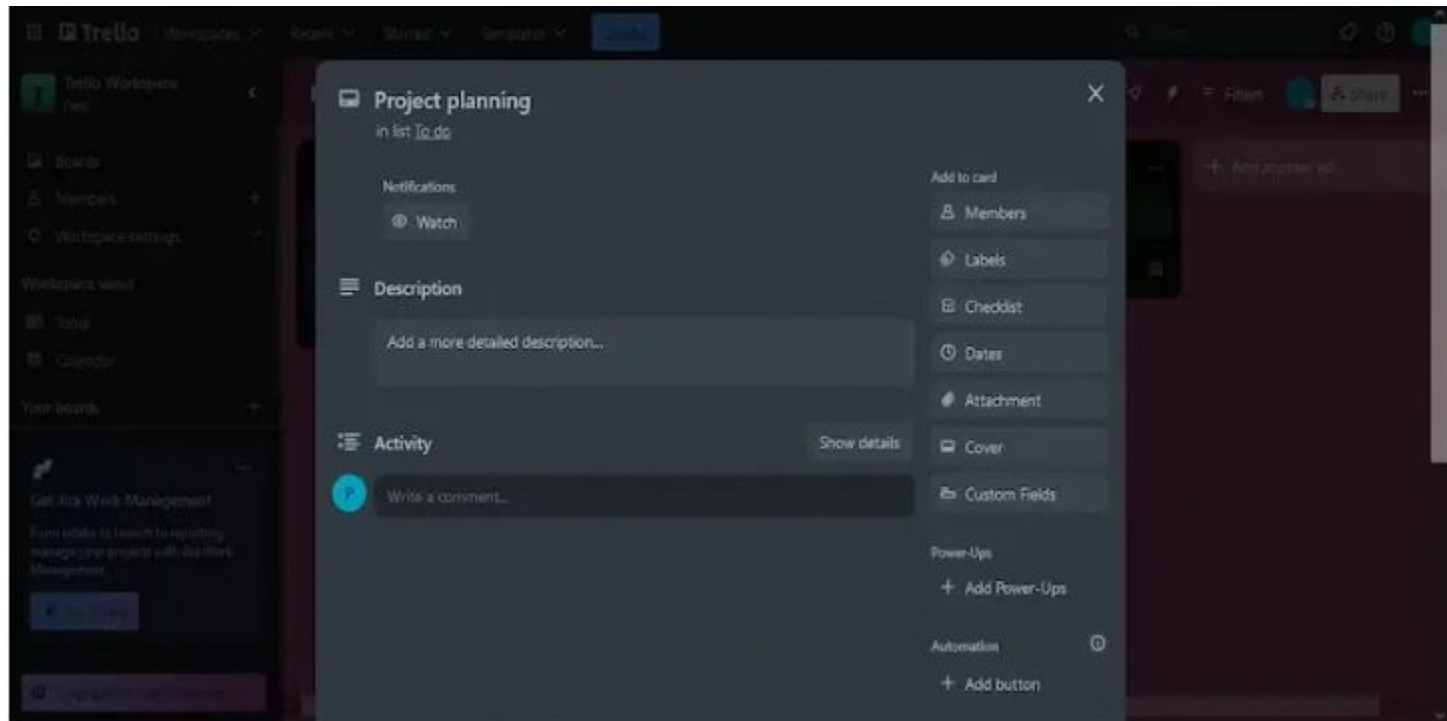


Exercise 1

1. **Sign Up and Create an Account:** Go to Trello's website and sign up for a free account.
2. **Create a Board:** Click on the "**Create new board**" button and give your board a name.
3. **Add Lists:** Inside your board, create lists to represent different stages of your workflow (e.g., To Do, In Progress, Done).
4. **Create Cards:** Add cards under each list for individual tasks. Click on a card to add details, due dates, attachments, and comments.
5. **Collaborate:** Invite team members by clicking on the "**Invite**" button and assigning them to specific cards.
6. **Move Cards:** Drag and drop cards between lists to reflect the progress of tasks.
7. **Use Power-Ups:** Enhance your board with **Power-Ups** (integrations and additional features) for added functionality.

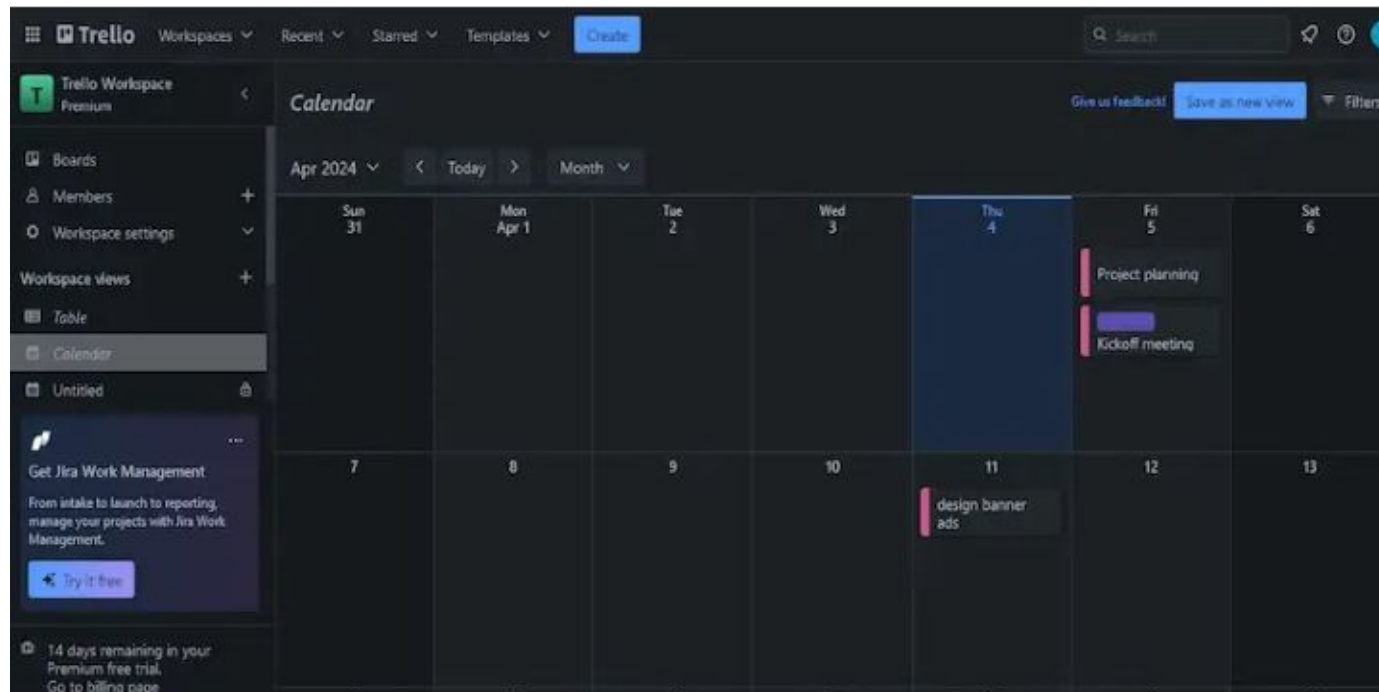
Using Trello Cards

Once the lists on the board are **completed**, cards need to be added. These Cards are the **building block of Trello** which provides details of the tasks within the project. Clicking a card expands to show detailed information like the example shown below.



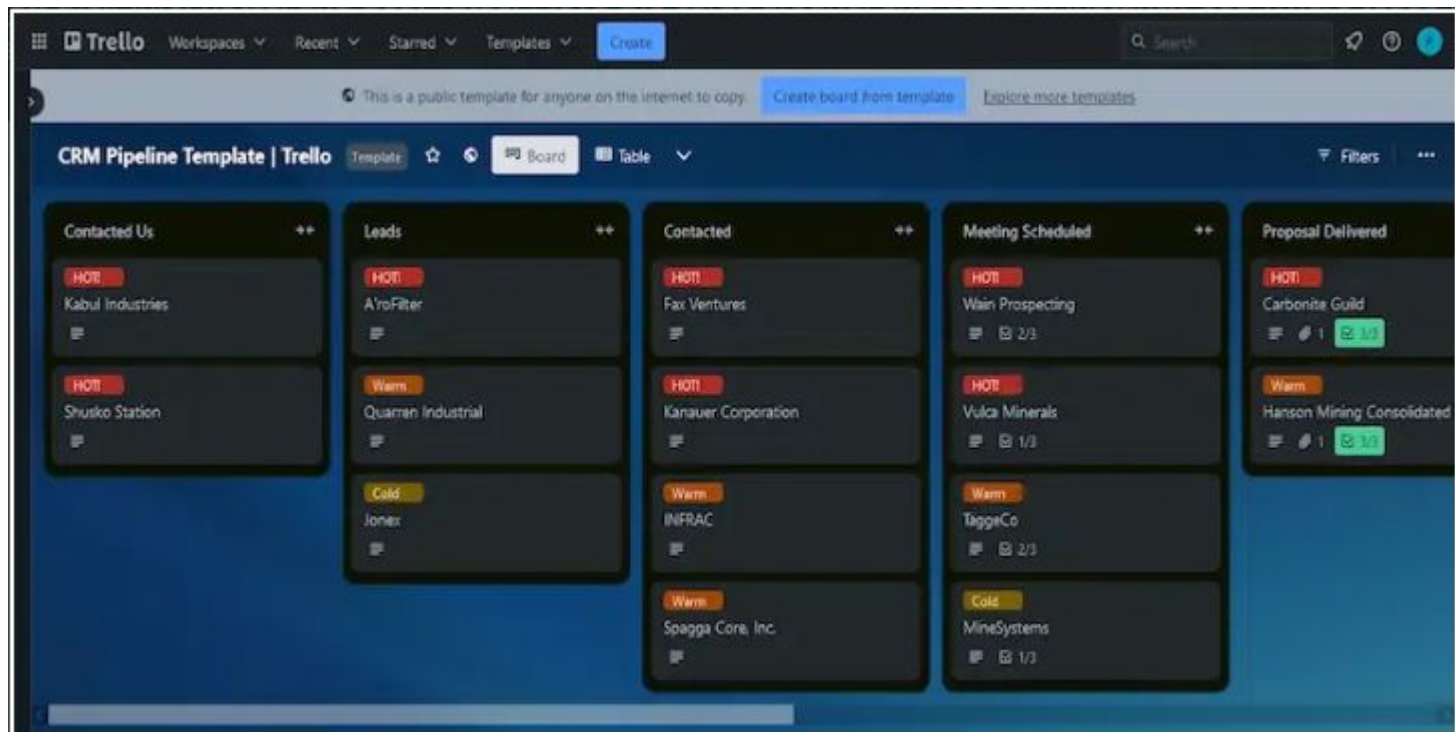
Calendar Planning

Trello's built-in view makes it a calendar planner, allowing you to see your deadlines visually, organized by due date. Lists act as timeframes ("**This Week**"), while color-coded labels highlight priorities. Optional Power-Ups further enhance functionality, letting you sync with external calendars. This makes Trello a powerful yet flexible way to manage your time, no matter how complex your schedule gets. By leveraging these features, you can effectively use Trello for project management, ensuring that all tasks are tracked and completed on time.

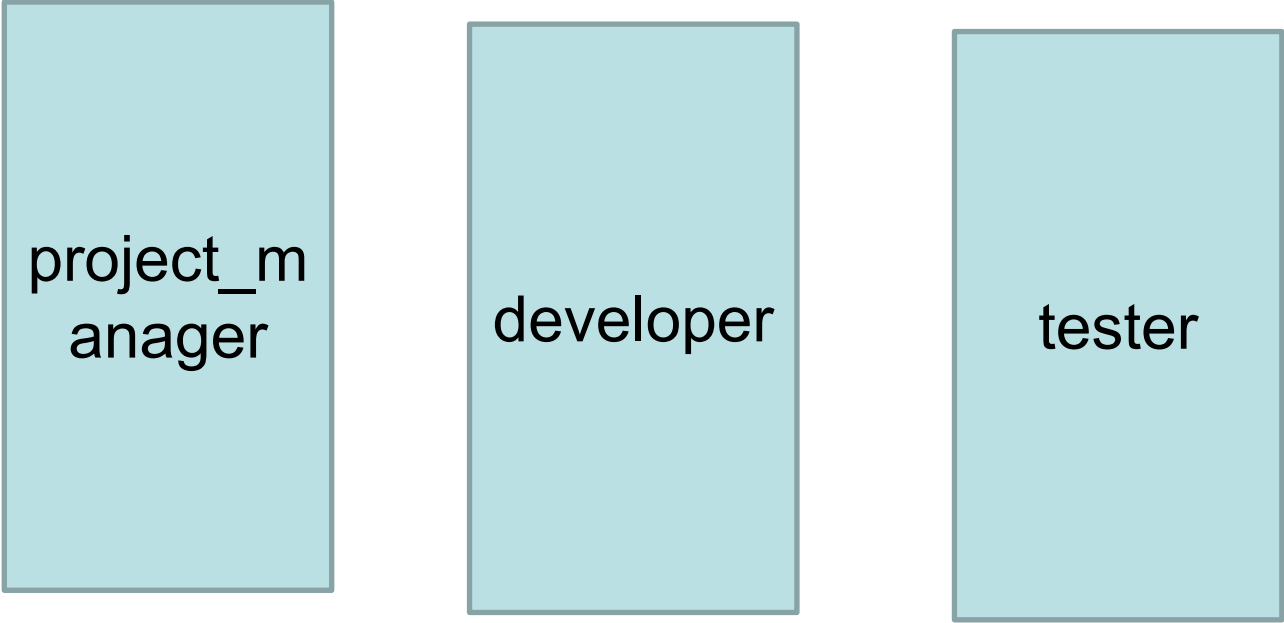


CRM Tool

Trello can be an essential tool in managing [Customer relationships](#). Each customer has a Trello card, similar to a digital folder holding all their info, notes, and deal value. Move the cards around, and you can instantly see where leads are within the pipeline - easy, at first glance, to understand what's in the queue for your team. And it's flexible for your team to work together on pushing these deals forward.



Python PM Tutorial 1 – Set 3 Roles and Permission



project_m
anager

developer

tester

Python PM Tutorial 1 – Set 3 Roles and Permission

```
class Project:
    def __init__(self, name):
        self.name = name
        self.tasks = []

    def add_task(self, task_name):
        self.tasks.append(task_name)
        print(f"Task '{task_name}' added to project '{self.name}'.")

    def view_tasks(self):
        print(f"Tasks in project '{self.name}':")
        for task in self.tasks:
            print(f"- {task}")

class Role:
    def __init__(self, name, permissions):
        self.name = name
        self.permissions = permissions

    def has_permission(self, permission):
        return permission in self.permissions
```

Python PM Tutorial 1 – Set 3 Roles and Permission

```
class User:
    def __init__(self, username, role, project):
        self.username = username
        self.role = role
        self.project = project

    def perform_action(self, action, task_name=None):
        if self.role.has_permission(action):
            if action == "add_task":
                self.project.add_task(task_name)
            elif action == "view_tasks":
                self.project.view_tasks()
            else:
                print("Action not supported.")
        else:
            print(f"User '{self.username}' does not have permission to '{action}'.")
```

Python PM Tutorial 1 – Set 3 Roles and Permission

Main Program

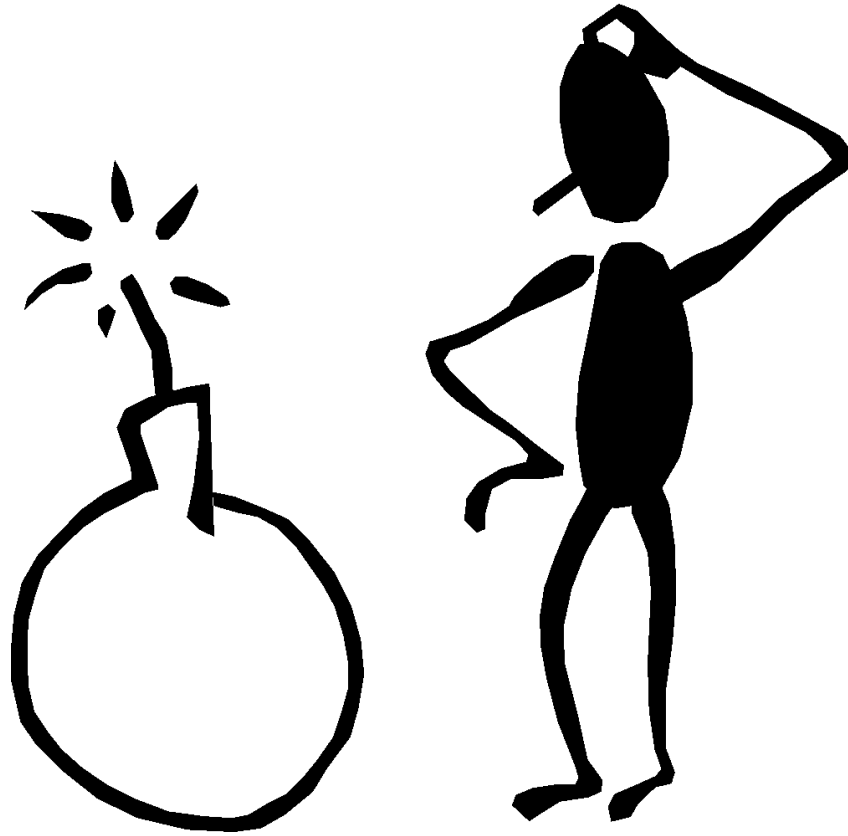
```
# Define roles and their permissions
developer_permissions = ["view_tasks", "add_task"] # Can view and add tasks
tester_permissions = ["view_tasks"] # Can only view tasks
project_manager_permissions = ["view_tasks", "add_task"] # Can view and add tasks

developer_role = Role("Developer", developer_permissions)
tester_role = Role("Tester", tester_permissions)
project_manager_role = Role("Project Manager", project_manager_permissions)

# Create a project
my_project = Project("Awesome Project")
# Create users and assign roles and the project
john = User("john_doe", developer_role, my_project)
jane = User("jane_smith", tester_role, my_project)
peter = User("peter_jones", project_manager_role, my_project)

john.perform_action("add_task", "Implement user authentication")
jane.perform_action("view_tasks")
peter.perform_action("add_task", "Design database schema")
john.perform_action("view_tasks")
jane.perform_action("add_task", "Write unit tests") # This will fail, as the tester doesn't have permission to add tasks
```

Project Risk Management



Project Risk – Definition

“Project risk is an uncertain event or condition that, if it occurs, has a positive or negative effect on a project objective”

“A combination of the probability of a defined **threat** or **opportunity (Likelihood)** and the magnitude of the consequences of the occurrence (Impact) defines a Risk Index”

Tasks	Threat	Reason
Data Collection	Delay 2 days	Business Scope Always changing
Data Cleaning	Delay 2 days	Business Scope Always changing
Machine Learning Model	Delay 0 days	Nil

Risk Impact

Threat → Scope → **Poor Quality Product**

Threat → Schedule → **Late Delivery**

Threat → Cost → **Overspend**

In addition there are health, safety and environmental threats that **must** be managed

Identify Risks

Assess likelihood and impact

Rank risks and prioritize

Define risk management approach & actions

Implement actions

Monitor & review

Make the management of risk integral to the way the project is managed

Ensure that cost and time contingencies are consistent with identified risks

Focus on the “significant few” – don’t try to manage too many risks

Be vigilant (be ready for possible problems and difficulties) and proactive

Project Monitoring and Control



Typical Monitoring Activities

- regular reviews of progress against schedule using WBS as basis (Plan against Baseline)
- regular review of actual costs (O/P from SAP) against budgeted costs and Earned Value at WBS level
- regular review of resource loading
- regular progress meetings with project team
- regular meetings with contractors
- production of periodic progress reports
- risk reviews
- inspections/ audits

Project Control

Typical Control Activities

- assign responsibilities at Work Package level
- staged authorisation of work to be done
- staged release of budgets (staged release of WBS(e) numbers)
- ensure PM has a 'Management Reserve' under his control
- seek corrective action reports when WPs go 'off track' (overrunning or overspending)
- release Management Reserve carefully



Project Monitoring and Control Summary

Monitor against the plan – status regularly

Take a factual approach to decisions

Identify management action early

Check that defined controls are being applied – correct if necessary

Apply change control

Automation on Stakeholder Engagement process

1. Identify Stakeholders

- Mapping Stakeholders: Begin by identifying all relevant stakeholders, both internal (employees, management) and external (customers, partners, investors). Create a comprehensive stakeholder map to visualize their roles and influence.
- Prioritization: Classify stakeholders based on their level of interest and influence. This will help prioritize engagement efforts and tailor strategies accordingly

2. Assess Needs

- Conduct Surveys and Interviews: Gather information on stakeholders' needs, expectations, and concerns through surveys or one-on-one interviews. This qualitative data is essential for understanding what stakeholders value most.
- Analyze Data: Use analytics tools to interpret the data collected. Identify common themes and specific requirements that can guide your engagement strategy.

3. Define Actions

- Develop Engagement Strategies: Based on the assessed needs, define specific actions tailored to each stakeholder group. This could include personalized communication plans, involvement in decision-making processes, or targeted information sessions.
- Set Clear Goals: Establish measurable objectives for each action to ensure accountability and track progress over time.

Automation on Stakeholder Engagement process

4. Establish Communication Channels

- Choose Appropriate Platforms: Select the most effective communication channels for engaging with different stakeholders. Options may include email newsletters, social media updates, webinars, or dedicated project management tools
- Automate Communication: Utilize automation tools to streamline communication efforts. For example, set up automated email campaigns or notifications to keep stakeholders informed without manual effort

5. Gather Feedback

- Implement Feedback Mechanisms: Create structured methods for collecting feedback from stakeholders after key interactions or milestones. This could involve follow-up surveys or feedback forms.
- Encourage Open Dialogue: Foster an environment where stakeholders feel comfortable sharing their thoughts and concerns. Use tools like online forums or chatbots to facilitate ongoing conversations.

6. Monitor and Review

- Track Engagement Metrics: Regularly monitor key performance indicators (KPIs) related to stakeholder engagement, such as response rates, satisfaction levels, and participation in events.
- Evaluate and Adjust Strategies: Conduct periodic reviews of your engagement strategies based on the feedback received and the metrics tracked. Make necessary adjustments to improve effectiveness and ensure alignment with stakeholder needs

Methodologies in Project Management

1. Waterfall Project Management
 - Waterfall project management follows a linear and sequential process where each phase must be completed before the next begins. It suits projects with fixed requirements and clear objectives.
2. Agile Project Management
 - Agile project management is an iterative approach that emphasizes flexibility, customer collaboration, and rapid delivery through short cycles called sprints, allowing teams to adapt quickly to changing requirements
3. Hybrid Project Management
 - Hybrid project management combines Agile and Waterfall methodologies, allowing teams to use structured planning while maintaining flexibility for iterative development, making it ideal for projects needing both predictability and adaptability.
4. Other Notable Methodologies
 - Other methodologies include Scrum, an Agile framework focusing on short cycles and team roles, and Lean, which maximizes value by minimizing waste and improving processes throughout the project lifecycle.

Successful Automation Projects: Case Studies

- **UiPath Automation Case Studies**

1. Enerjisa (Energy): Improved internal processes and customer service, leading to enhanced operational efficiency and customer satisfaction.
2. Suncoast Credit Union (Financial Services): Utilized AI and automation to improve member trust and accelerate growth, streamlining banking processes.
3. MongoDB (Technology): Saved over 150,000 working hours and \$1.5 million by automating time-consuming data management tasks.

- **Kawasaki Robotics Case Studies**

1. German Brewery (Food & Beverage): Implemented robotic palletizing systems to reduce labor costs and increase production capacity.
2. Automated Bottle Production Line (Food & Beverage): Enhanced material handling and palletizing processes, resulting in streamlined operations and reduced production times.

- **Ittransition RPA Use Cases**

1. Heritage Bank (Banking): Automated 80 processes across departments, significantly improving operational efficiency and reducing customer interaction processing times.
2. Thermo Fisher Scientific (Biotechnology): Achieved a 70% reduction in invoice processing time for 824,000 invoices annually, enhancing accuracy in financial operations.

Successful Automation Projects: Case Studies

- **Nividous RPA Case Studies**

1. **Manufacturer Invoice Processing:** Automated data extraction from invoices, saving over \$90,000 annually and speeding up processing times
2. **Healthcare Patient Claims Processing:** Reduced handling time by 70% through automation of data extraction and claims submission, improving cash flow management.

- **Industrial Automation Insights**

1. **JR Automation & Advanced Drainage Systems (Manufacturing):** Developed a flexible pipe sorting system that improved operational efficiency while reducing manual labor needs.
2. **Snowboard Manufacturing Digital Transformation:** Streamlined manufacturing processes using automation technologies, increasing productivity and reducing time-to-market.

Benefits of Using Automation in Project Management

1. Establishing Risks to be Managed

- Real-Time Risk Monitoring: Automation allows for continuous monitoring of project parameters, enabling teams to identify potential risks as they arise rather than after the fact. This proactive approach enhances the ability to mitigate risks effectively before they escalate
- Data Analysis and Predictive Insights: Automated systems can analyze large volumes of data to identify trends and patterns that may indicate emerging risks. This predictive capability allows project managers to prepare for potential challenges and implement strategies accordingly

2. Establishing Costs and Durations

- Accurate Cost Estimation: Automation tools can analyze historical data and current project metrics to provide more accurate cost estimations. This helps in budgeting and financial planning by reducing guesswork and improving reliability
- Streamlined Reporting: Automated reporting features generate cost and duration reports quickly, providing stakeholders with timely insights into project status without manual compilation efforts. This enhances transparency and facilitates informed decision-making

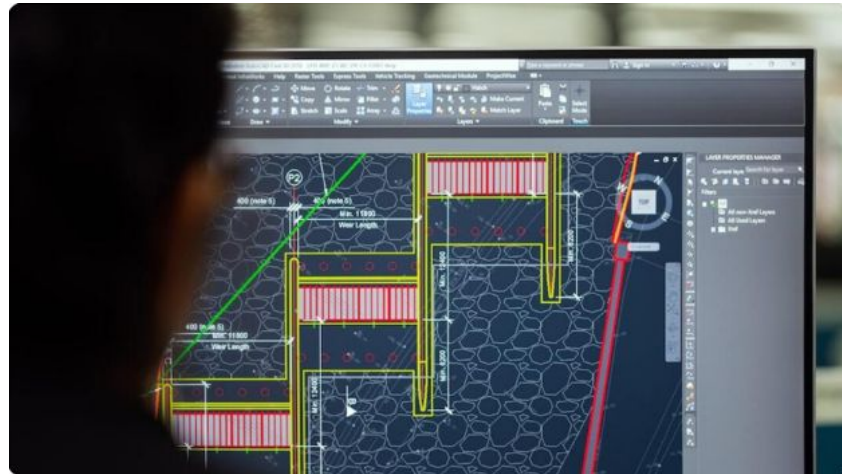
3. Establishing Resources Required

- Resource Optimization: Automation helps in tracking resource allocation and utilization across projects. By analyzing resource usage patterns, organizations can optimize their workforce and materials, ensuring that resources are used efficiently
- Scalability: Automated systems can easily scale to accommodate changes in resource requirements as projects grow or shrink. This adaptability ensures that organizations do not overcommit or underutilize their resources

Understanding Business Process Automation

Defining the Future of Efficiency

- **Definition:** Business process automation (BPA) refers to the use of technology to automate repetitive, manual tasks in business operations to increase efficiency and accuracy.
- **Benefits:** The advantages of BPA include reduced operational costs, increased speed and accuracy of processes, and the ability to reallocate human resources to more strategic tasks.
- **Tools:** Various tools such as workflow automation software, robotic process automation, and AI are instrumental in facilitating BPA by removing bottlenecks in business operations.
- **Examples:** Common applications include automating invoice processing, customer relationship management (CRM), and supply chain management, demonstrating BPA's versatility across industries.



Integration of Project Management and Business Process Automation

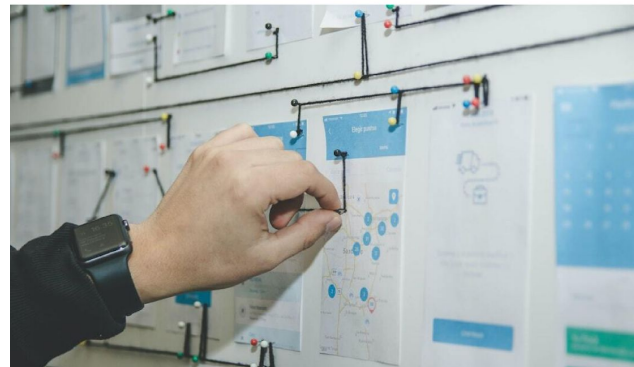
Creating Cohesion for Success

Scope	Description
Synergy	The integration of project management with BPA leads to improved planning, execution, and monitoring of initiatives, enhancing overall project success.
Process Mapping	Visualizing workflows and processes aids in identifying inefficiencies and opportunities for automation, aligning project management initiatives with business objectives.
Best Practices	Employing best practices such as stakeholder engagement, risk management, and iterative development can significantly improve project outcomes in automation.
Alignment	Ensuring that project management frameworks align with automation goals fosters a culture of continuous improvement and operational excellence within the organization.

Integration of Project Management and Business Process Automation

Understanding the Fundamentals

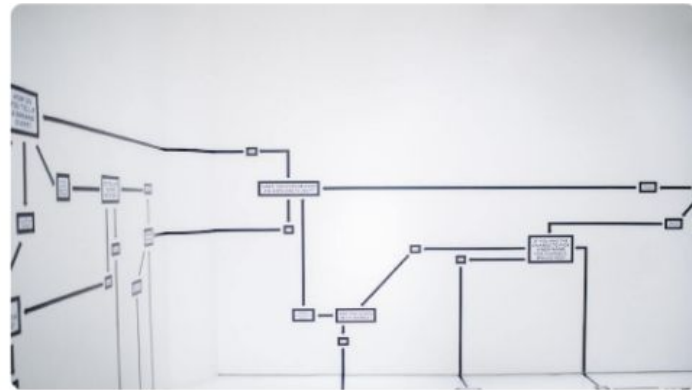
- **Definition:** Project management is the discipline of planning, organizing, and overseeing the successful execution of projects in the context of business process automation, which aims to optimize workflows and improve efficiency.
- **Importance:** The significance of project management in business process automation lies in its ability to streamline processes, enhance productivity, and ensure resources are effectively utilized to meet business objectives.
- **Goals:** The primary objectives encompass enhancing efficiency, reducing costs, improving quality and ensuring successful project delivery that meets stakeholder expectations.
- **Overview:** An effective project management approach integrates all phases of automation projects, from inception through to evaluation, thus providing a comprehensive roadmap for implementation.



Key Steps in Project Management

The Phases of Project Execution

- Initiation: Defining the project scope, objectives, and stakeholders to establish a shared understanding of the project goals and outcomes.
- Planning: Creating a detailed project plan that outlines tasks, timelines, resources, and budget, ensuring a clear pathway toward project completion.
- Execution: Implementing the project plan, coordinating teams, and managing resources to deliver the project output effectively.
- Monitoring: Continuous oversight of project progress, ensuring alignment with goals, addressing any deviations, and making necessary adjustments to stay on track.
- Closing: Finalizing all project elements, conducting evaluations, and documenting lessons learned to inform future projects and ensure comprehensive closure.



Project Management Tasks Priority Tutorial

We are going to conduct an booking management System. In working out the project, we apply python to create 2 sprint within 2 weeks that include 8 tasks

```
# First install required packages
```

```
import pandas as pd
```

```
import os
```

```
import plotly.express as px
```

```
from datetime import datetime
```

```
CSV_FILE = "bookingsystem_tasks.csv"
```

```
# Initial sample tasks for an booking management system
```

```
tasks = [
```

```
    # Sprint 1 - Core Features
```

```
    {'Task': 'User Authentication', 'Start': '2023-01-01', 'Finish': '2023-01-05', 'Priority': 'Must-Have', 'Sprint': 1},
```

```
    {'Task': 'Property Listing CRUD', 'Start': '2023-01-03', 'Finish': '2023-01-07', 'Priority': 'Must-Have', 'Sprint': 1},
```

```
    {'Task': 'Booking System Core', 'Start': '2023-01-06', 'Finish': '2023-01-10', 'Priority': 'Must-Have', 'Sprint': 1},
```

```
    {'Task': 'Basic Search Filters', 'Start': '2023-01-08', 'Finish': '2023-01-12', 'Priority': 'Should-Have', 'Sprint': 1},
```

```
    # Sprint 2 - Enhanced Features
```

```
    {'Task': 'Payment Integration', 'Start': '2023-01-15', 'Finish': '2023-01-19', 'Priority': 'Must-Have', 'Sprint': 2},
```

```
    {'Task': 'Review System', 'Start': '2023-01-17', 'Finish': '2023-01-21', 'Priority': 'Should-Have', 'Sprint': 2},
```

```
    {'Task': 'Recommendation Engine', 'Start': '2023-01-20', 'Finish': '2023-01-24', 'Priority': 'Could-Have', 'Sprint':
```

```
2},
```

```
    {'Task': 'Social Media Login', 'Start': '2023-01-22', 'Finish': '2023-01-26', 'Priority': 'Could-Have', 'Sprint': 2}
```

```
]
```

Project Management Tasks Priority Tutorial

(Continue)...

We define function that save all tasks into csv file named “bookingsystem_tasks.csv”

```
def save_tasks_to_csv(tasks, csv_file):
    df = pd.DataFrame(tasks)
    if not os.path.exists(csv_file):
        df.to_csv(csv_file, index=False)
        print(f"Created new CSV file with sample tasks at {csv_file}")
    else:
        print(f"CSV file already exists at {csv_file}. Overwriting...")
        df.to_csv(csv_file, index=False)
        print(f"Updated tasks saved to {csv_file}")
```

#Main Program

```
save_tasks_to_csv(tasks, CSV_FILE)
```

Task	Start	Finish	Priority	Sprint
User Authentication	1/1/2023	5/1/2023	Must-Have	1
Property Listing CRUD	3/1/2023	7/1/2023	Must-Have	1
Booking System Core	6/1/2023	10/1/2023	Must-Have	1
Basic Search Filters	8/1/2023	12/1/2023	Should-Have	1
Payment Integration	15/1/2023	19/1/2023	Must-Have	2
Review System	17/1/2023	21/1/2023	Should-Have	2
Recommendation Engine	20/1/2023	24/1/2023	Could-Have	2
Social Media Login	22/1/2023	26/1/2023	Could-Have	2

```
import pandas as pd
bookingsystem = pd.read_csv("bookingsystem_tasks.csv")
bookingsystem.head(5)
```

Project Management Tasks Priority Tutorial

Task 2 : We are going to generate and display gantt chart based on previous tasks.

```
csv_file = "bookingsystem_tasks.csv"
```

```
#define function to load tasks from csv
```

```
def load_tasks_from_csv(csv_file):
```

```
    if not os.path.exists(csv_file):
```

```
        print(f"Error: CSV file not found at {csv_file}. Please run save_tasks.py first.")
```

```
        return None
```

```
    else:
```

```
        df = pd.read_csv(csv_file)
```

```
        df['Start'] = pd.to_datetime(df['Start'])
```

```
        df['Finish'] = pd.to_datetime(df['Finish'])
```

```
        return df
```

Project Management Tasks Priority Tutorial

(Continue)... Here we define function that generate gantt chart and save as png

```
def display_gantt_chart(df):  
    # Create a figure with a secondary axis for the Gantt chart  
    import plotly.graph_objects as go  
    fig = go.Figure()  
  
    # Define colors for priorities  
    color_map = {  
        'Must-Have': '#FF4C4C', # Red - Critical  
        'Should-Have': '#FFA500', # Orange - Important  
        'Could-Have': '#4CAF50' # Green - Nice-to-have  
    }  
  
    # Iterate over each task and add it to the Gantt chart  
    for index, row in df.iterrows():  
        fig.add_trace(go.Scatter(  
            x=[row['Start'], row['Finish']],  
            y=[index, index],  
            mode='lines',  
            line_shape='hv',  
            line_color=color_map[row['Priority']],  
            hoverinfo='text',  
            hovertext=f"Task: {row['Task']}, Priority: {row['Priority']}" #or  
            #hovertext=f"Priority: {row['Priority']}"  
        ))
```

Project Management Tasks Priority Tutorial

(Continue)... Here we define function that generate gantt chart and save as png
#for index, row in df.iterrows()

Set layout properties

```
fig.update_layout(  
    title="Housing Booking System Project Schedule",  
    xaxis_title="Date",  
    yaxis_title="Task",  
    yaxis=dict(  
        tickmode='array',  
        tickvals=list(range(len(df))),  
        ticktext=df['Task']  
    ),  
    xaxis=dict(  
        type='date'  
    )  
)
```

```
fig.show()
```

Project Management Tasks Priority Tutorial

(Continue)... Main Program to run function to generate chart

```
#Display Gantt Chart
```

```
df = load_tasks_from_csv(CSV_FILE)
```

```
if df is not None:
```

```
    display_gantt_chart(df)
```





HKUSPACE
香港大學專業進修學院
HKU School of Professional and Continuing Education

THANK YOU

